

분산처리

Part II: Locality, Communication, and Contention



Computer & Ai

Department of Computer Engineering

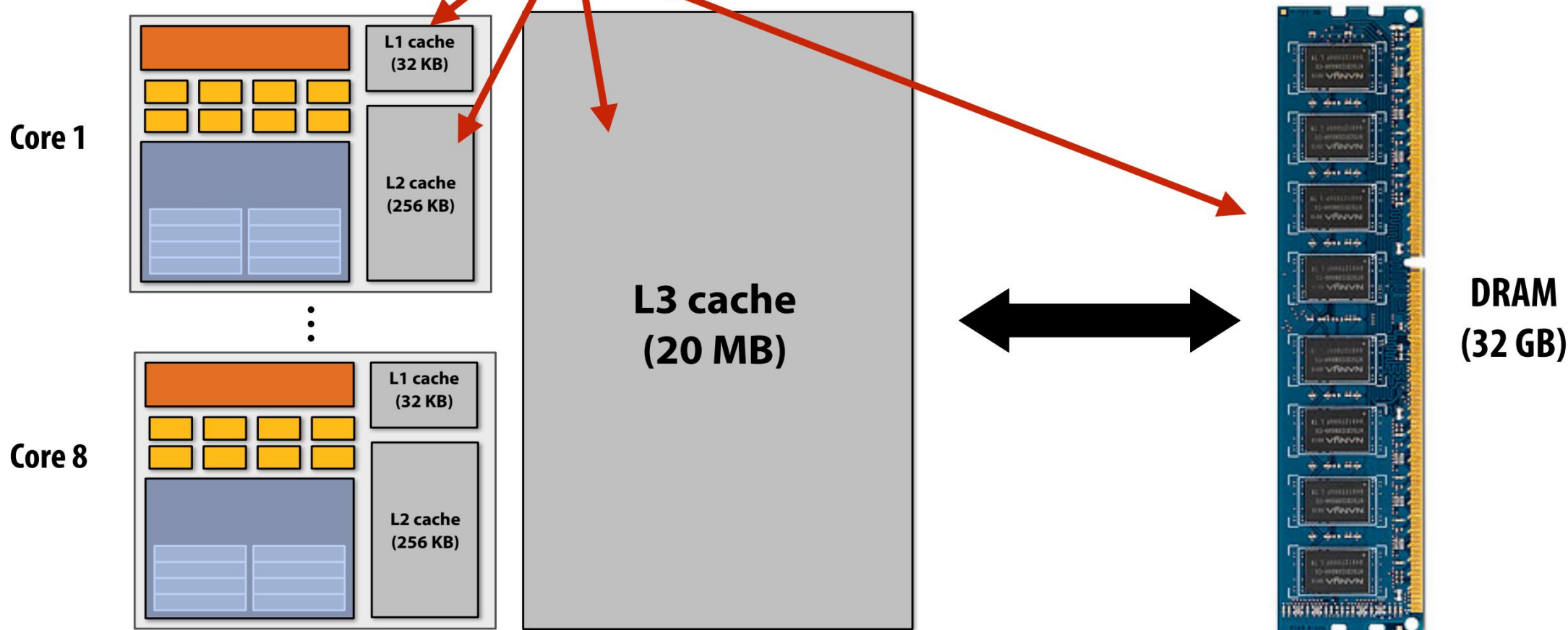
- Techniques for reducing the costs of communication
 - Between processors
 - Between processor(s) and memory
- General program optimization tips

지금까지 학습한 내용에서는 모든 프로세서가 단일 공유 주소 공간의 추상화를 제공하는 메모리 시스템에 연결되어 있다고 가정했다.

하지만 이러한 추상화를 구현하는 것은 상당히 복잡할 수 있다.

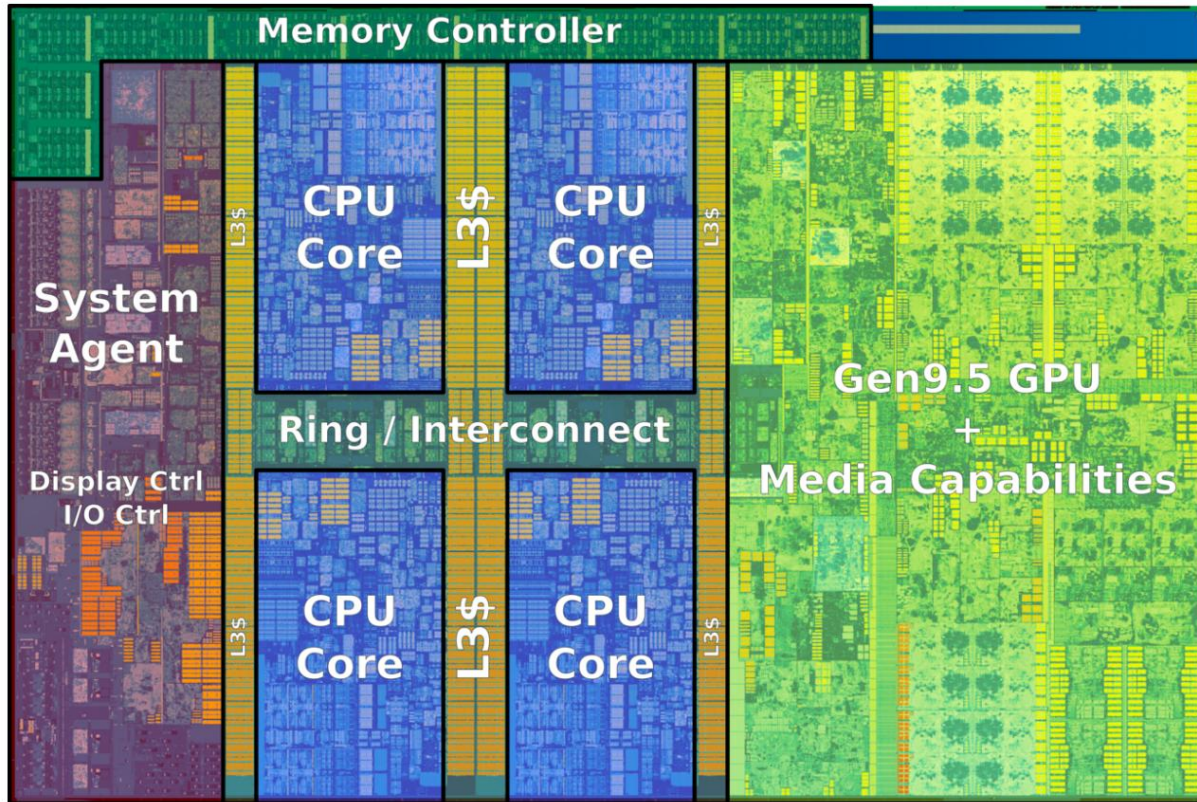
최신 컴퓨터에서 선형 메모리 주소 공간을 추상화하는 것은 복잡

“주소 X에 저장된 값을 레지스터 R0에 로드하라”는 명령에는 여러 데이터 캐시에 의한 복잡한 연산 시퀀스와 DRAM에 대한 액세스가 포함될 수 있다.

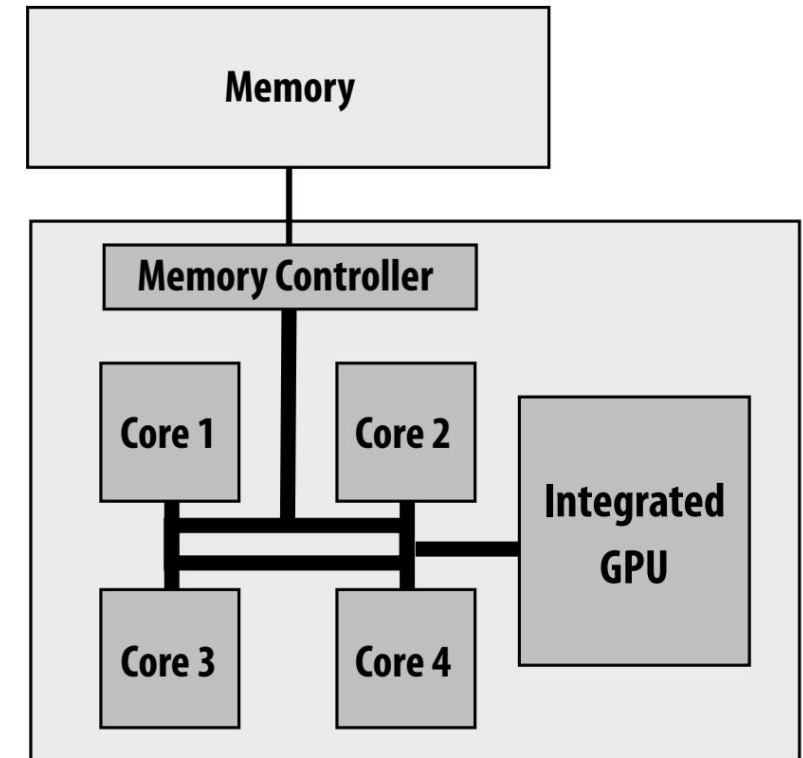


Shared address space hardware architecture

- 모든 프로세서는 모든 메모리 위치를 직접 참조할 수 있다.



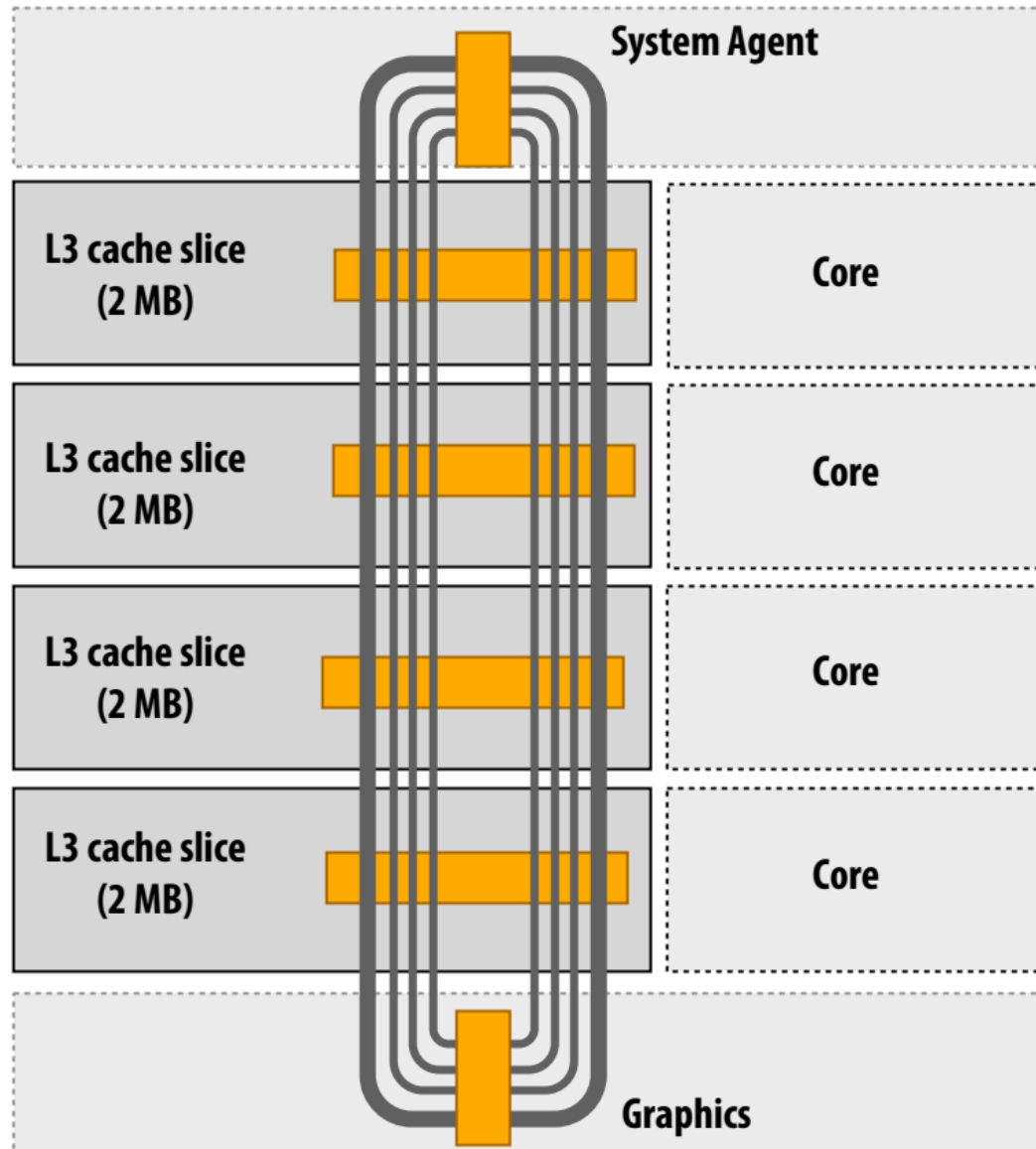
Example: Intel Core i7 processor (Kaby Lake)



Intel Core i7 (quad core)
(interconnect is a ring)

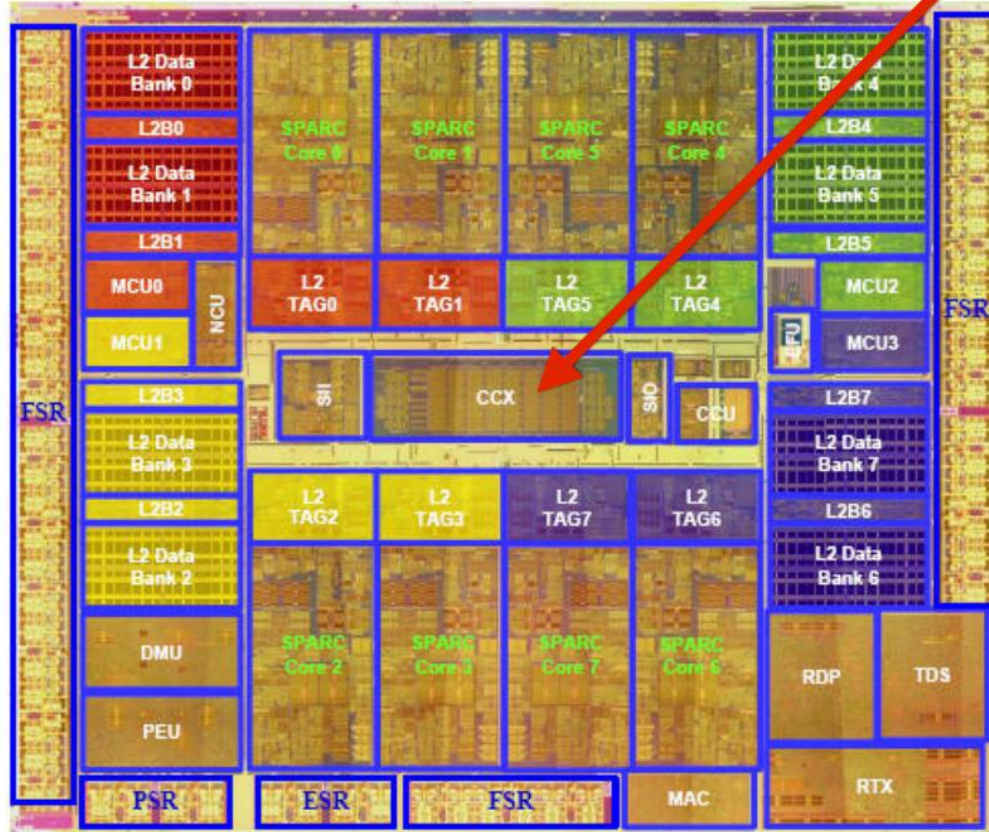
Intel's ring interconnect

Sandy Bridge microarchitecture에 도입



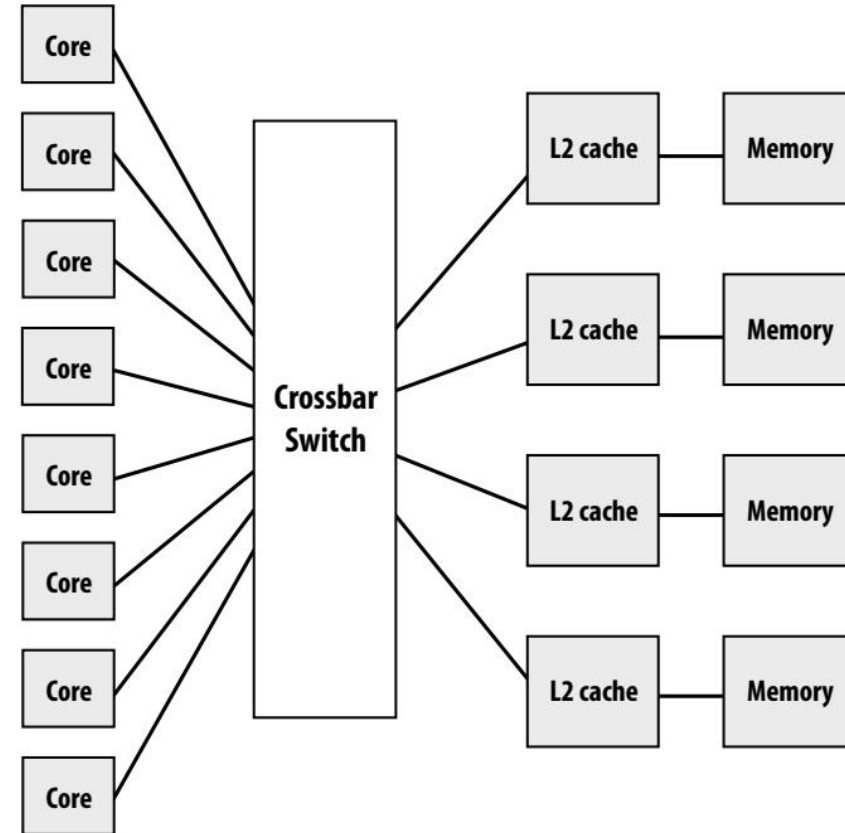
- 4개의 링: 다양한 유형의 메시지용
 - request
 - snoop
 - ack
 - data (32 bytes)
- 6개의 상호 연결 노드: 4개의 L3 캐시 "의 슬라이스" + 시스템 에이전트 + 그래픽
- 링 버스에 두 번 연결된 L3의 각 뱅크
- 3.4GHz ~ 435GB/sec에서 코어에서 L3까지의 이론적 피크 BW
 - 각 코어가 로컬 슬라이스에 액세스하는 경우

SUN Niagara 2 (UltraSPARC T2): crossbar interconnect`



Eight core processor

**Note area of crossbar (CCX):
about same area as one core on chip**



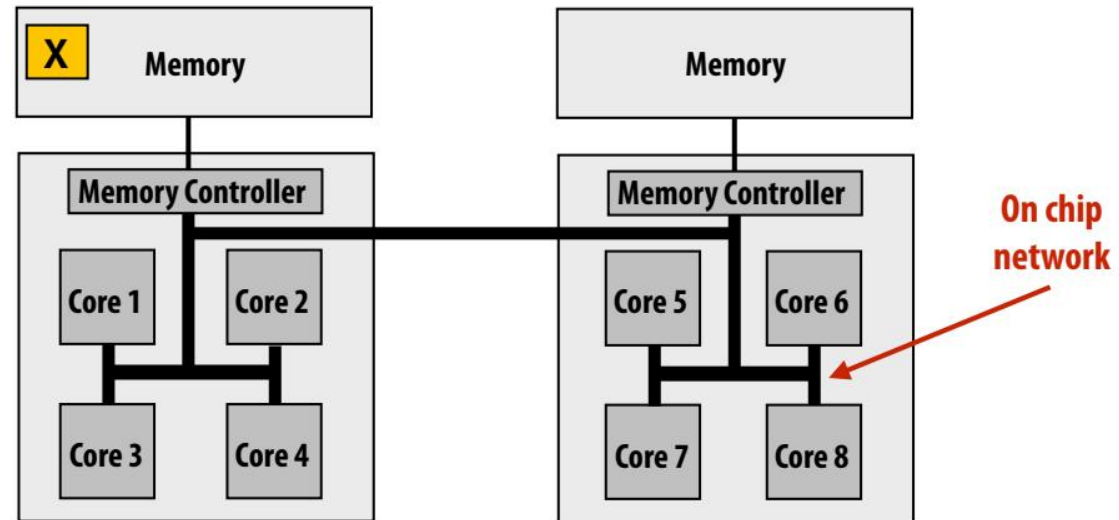
Crossbar = All cores connected directly to all others

Non-uniform memory access (NUMA)

- 메모리 위치에 액세스하는 지연 시간은 시스템의 처리 코어에 따라 다를 수 있습니다.
- 어떤 한 위치의 대역폭도 CPU 코어에 따라 다를 수 있습니다*.



Example: modern multi-socket configuration



- * 실제로는 단일 소켓 시스템에서도 NUMA 동작을 확인할 수 있다 (캐시 슬라이스마다 각 코어와의 거리가 다를 수 있음).

Summary: shared address space model

▪ 통신 추상화(Communication abstraction)

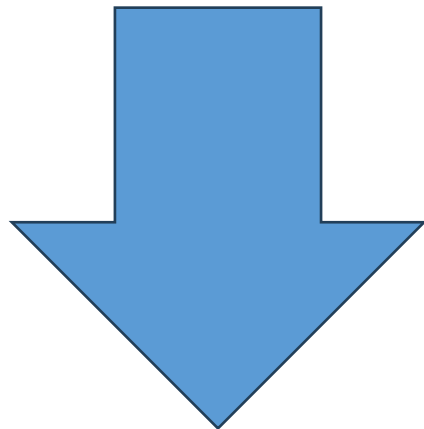
- 스레드는 공유 주소 공간에서 변수를 읽고 씁니다.
- 스레드는 동기화 기본 요소(잠금, 원자 연산 등)를 조작합니다.
- 단일 프로세서 프로그래밍의 논리적 확장*

▪ 효율적인 구현을 위해 하드웨어 지원 필요

- 모든 프로세서가 모든 주소에서 로드 및 저장 가능
- 많은 수의 프로세서로 확장하는 데 비용이 많이 들 수 있음
(코어 수가 많은 프로세서가 비싼 이유 중 하나)

* 하지만 NUMA를 구현하려면 성능 최적화를 위해 로컬리티에 대한 추론이 필요합니다.

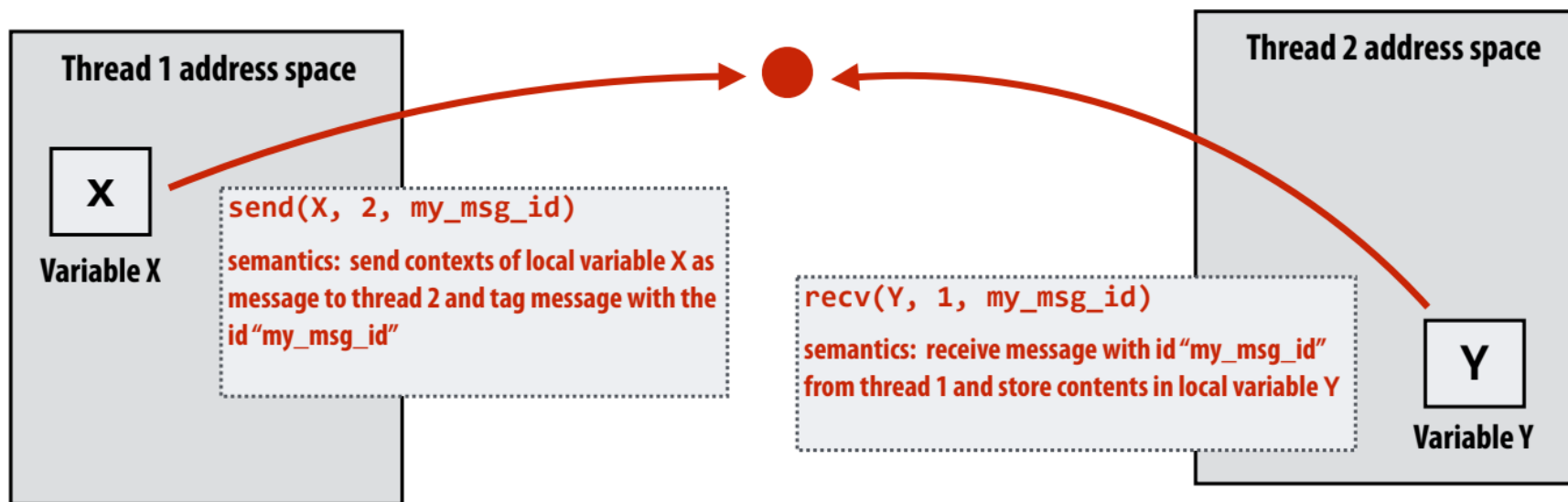
- 공유 주소 공간 모델에서 스레드는 공유 주소 공간에 있는 변수를 읽고 쓰는 방식으로 통신.
- 프로세서 간의 통신을 보다 명확하게 하는 다른 추상화를 고려해 보자



Message passing

Message passing model (abstraction)

- 스레드는 고유한 사적 주소 공간(private address spaces) 내에서 작동.
- 스레드는 메시지를 송수신하여 통신.
 - send: 수신자, 전송할 버퍼, 선택적 메시지 식별자("태그")를 지정.
 - 수신: 발신자, 데이터를 저장할 버퍼 및 선택적 메시지 식별자 지정
 - 메시지 전송은 스레드 1과 2 간에 데이터를 교환할 수 있는 유일한 방법.



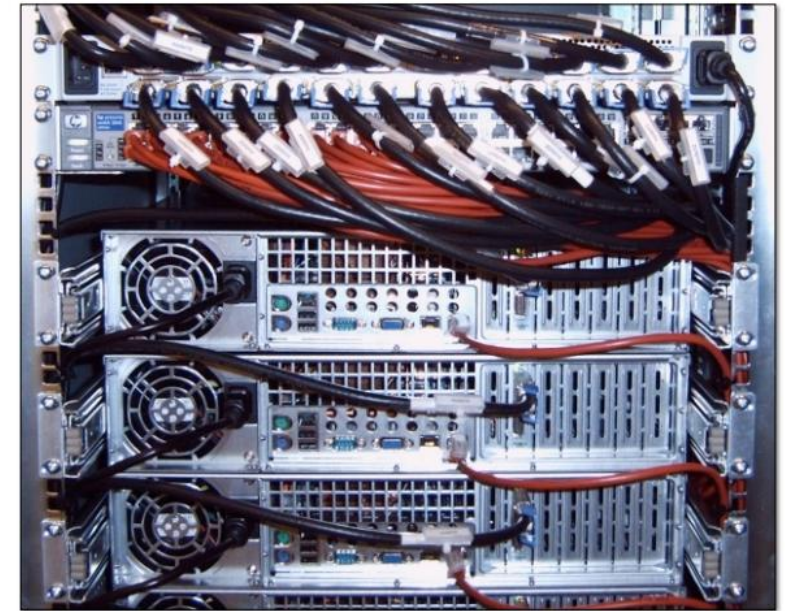
(Communication operations shown in red)

일반적인 비유: 우체통 메일



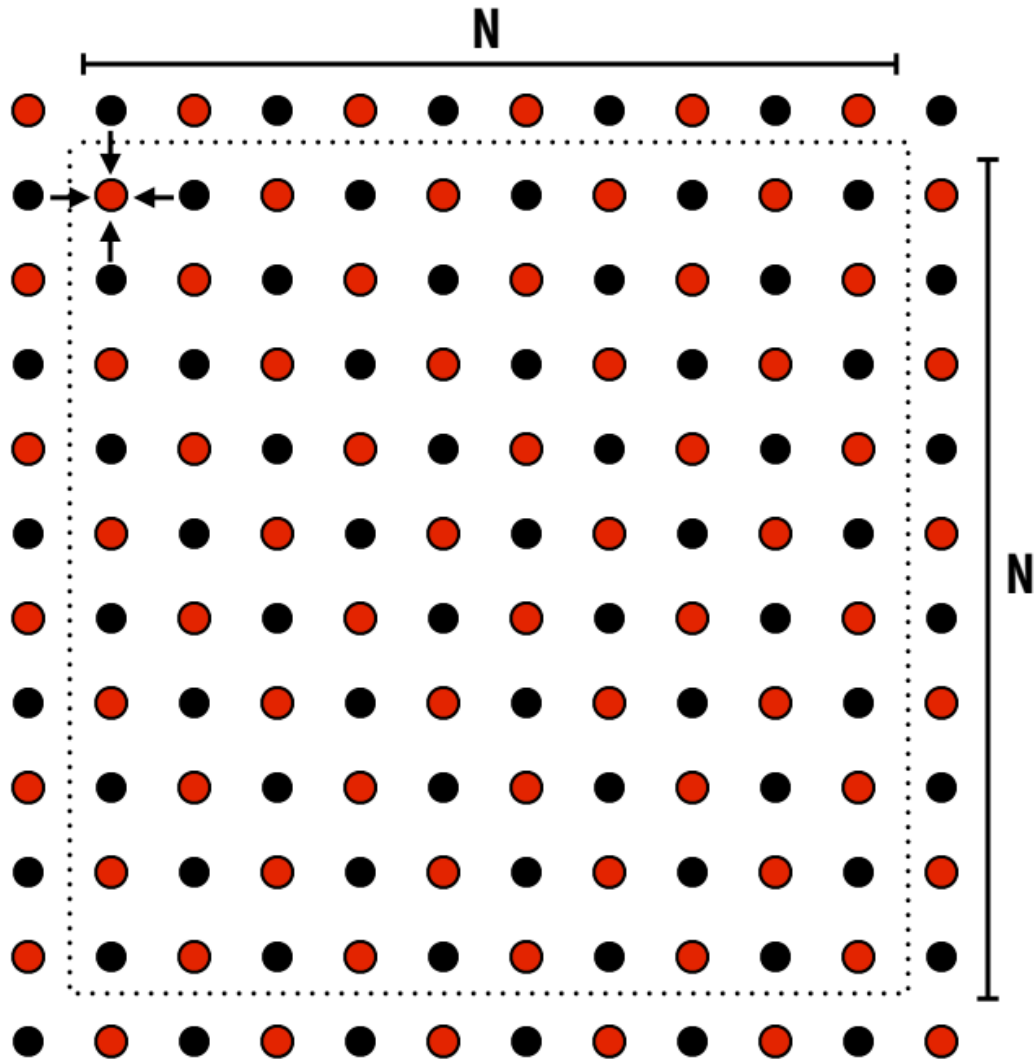
Message passing (implementation)

- 하드웨어는 모든 프로세서에 대해 단일 공유 주소 공간을 구현할 필요가 없음
(노드 간 메시지 통신 메커니즘만 제공하면 됨).
- 상품 시스템을 함께 연결하여 대규모 병렬 시스템을 구성할 수 있습니다.
(메시지 전달은 클러스터와 슈퍼컴퓨터를 위한 프로그래밍 모델입니다.)



Cluster of workstations
(Infiniband network)

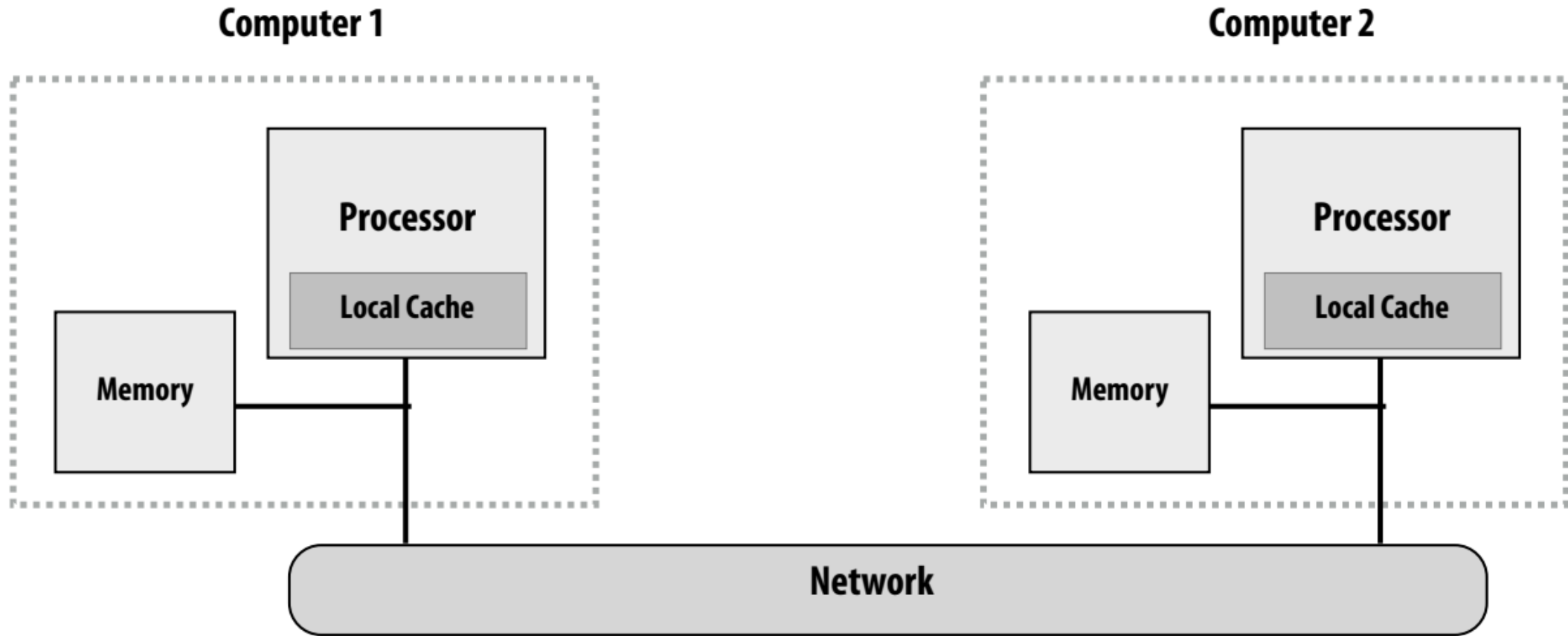
Message passing expression of solver



- 그리드 솔버 애플리케이션을 Recall
- 모든 빨간색 셀을 병렬로 업데이트.
- 적색 셀 업데이트가 완료되면 모든 검정색 셀을 병렬로 업데이트합니다(적색 셀에 대한 종속성 반영).
- 수렴할 때까지 반복.

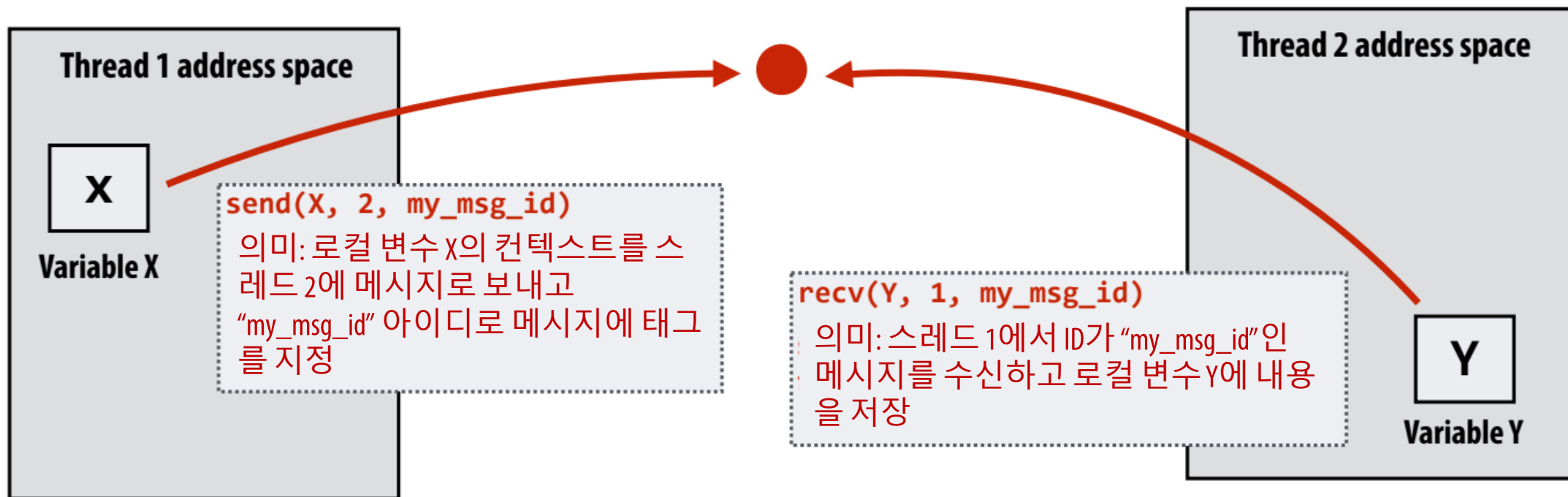
메시지를 통한 통신으로 병렬 그리드 솔버를 표현(샘플)

메시지 전달 머신 구성의 한 가지 가능성: 두 대의 머신으로 구성된 클러스터



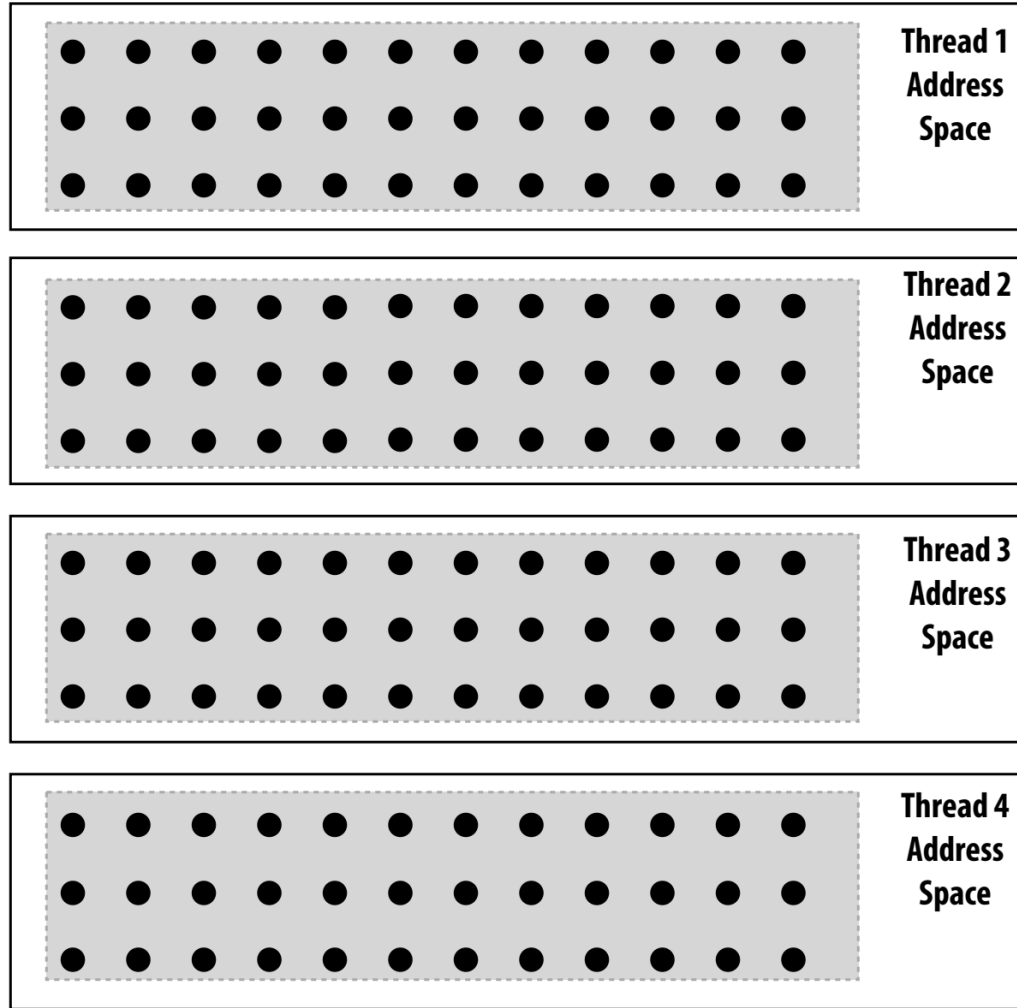
Review: Message passing model

- 스레드는 고유한 사적 주소 공간(private address spaces) 내에서 작동.
- 스레드는 메시지를 송수신하여 통신.
 - send: 수신자, 전송할 버퍼, 선택적 메시지 식별자("태그")를 지정.
 - 수신: 발신자, 데이터를 저장할 버퍼 및 선택적 메시지 식별자 지정
 - 메시지 전송은 스레드 1과 2 간에 데이터를 교환할 수 있는 유일한 방법.



(Communication operations shown in red)

메시지 전달 모델: 각 스레드는 자체 주소 공간에서 작동.

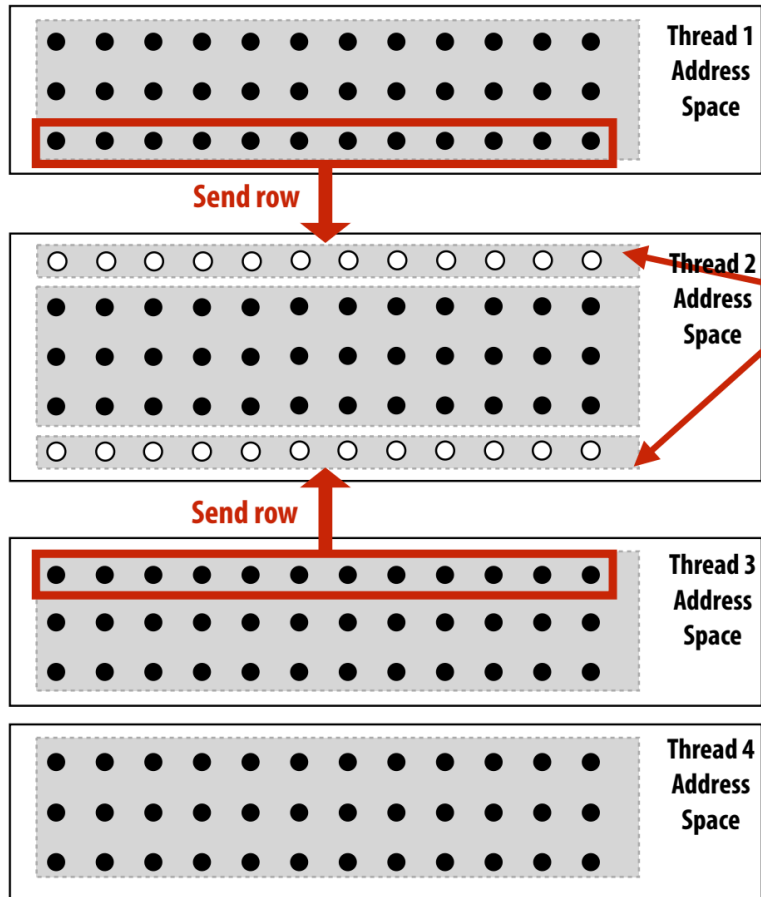


이 그림: 4개의 스레드

그리드 데이터는 4개의 할당으로 분할되며,
각각 4개의 고유 스레드 주소 공간(스레드당 4개의 프
라이빗 배열) 중 하나에 상주

이제 프로그램을 올바르게 실행하려면 데이터 복제가 필요함

4개의 개별 주소 공간(4개의 프라이빗 배열)에 저장된 그리드 데이터



예시:

적색 셀 처리가 완료되면 스레드 1과 스레드 3이 한 행의 데이터를 스레드 2로 전송 (스레드 2는 다음 단계에서 검정 셀을 업데이트하기 위해 최신 적색 셀 정보가 필요함).

“고스트 셀(Ghost cells)”은 원격 주소 공간에서 복제된 그리드 셀.
고스트 셀의 정보는 다른 스레드가 '소유'한다고 말하는 것이 일반적임.

Thread 2 logic:

```
float* local_data = allocate(N+2, rows_per_thread+2);
```

```
int tid = get_thread_id();  
int bytes = sizeof(float) * (N+2);
```

```
// receive ghost row cells (white dots)  
recv(&local_data[0], bytes, tid-1);  
recv(&local_data[rows_per_thread+1], bytes, tid+1);
```

```
// Thread 2 now has data necessary to perform  
// its future computation
```


Message passing solver

- 공유 주소 공간 솔버와 유사한 구조이지만 이제 메시지 송수신에 명시적인 통신이 가능

“neighbor threads”로 고스트 행(ghost rows) 보내기 및 받기

계산 수행(솔버의 공유 주소 공간 버전에서와 마찬가지로)

모든 스레드는 로컬 my_diff를 스레드 0으로 전송

스레드 0은 전역 차이(global dif)를 계산하고 종료 근거를 평가한 후 결과를 다른 모든 스레드로 다시 보냄.

```
int N;
int tid = get_thread_id();
int rows_per_thread = N / get_num_threads();

float* localA = allocate(rows_per_thread+2, N+2);

// assume localA is initialized with starting values
// assume MSG_ID_ROW, MSG_ID_DONE, MSG_ID_DIFF are constants used as msg ids
////////////////////////////////////

void solve() {
    bool done = false;
    while (!done) {

        float my_diff = 0.0f;

        if (tid != 0)
            send(&localA[1,0], sizeof(float)*(N+2), tid-1, MSG_ID_ROW);
        if (tid != get_num_threads()-1)
            send(&localA[rows_per_thread,0], sizeof(float)*(N+2), tid+1, MSG_ID_ROW);

        if (tid != 0)
            recv(&localA[0,0], sizeof(float)*(N+2), tid-1, MSG_ID_ROW);
        if (tid != get_num_threads()-1)
            recv(&localA[rows_per_thread+1,0], sizeof(float)*(N+2), tid+1, MSG_ID_ROW);

        for (int i=1; i<rows_per_thread+1; i++) {
            for (int j=1; j<N+1; j++) {
                float prev = localA[i,j];
                localA[i,j] = 0.2 * (localA[i-1,j] + localA[i,j] + localA[i+1,j] +
                                     localA[i,j-1] + localA[i,j+1]);
                my_diff += fabs(localA[i,j] - prev);
            }
        }

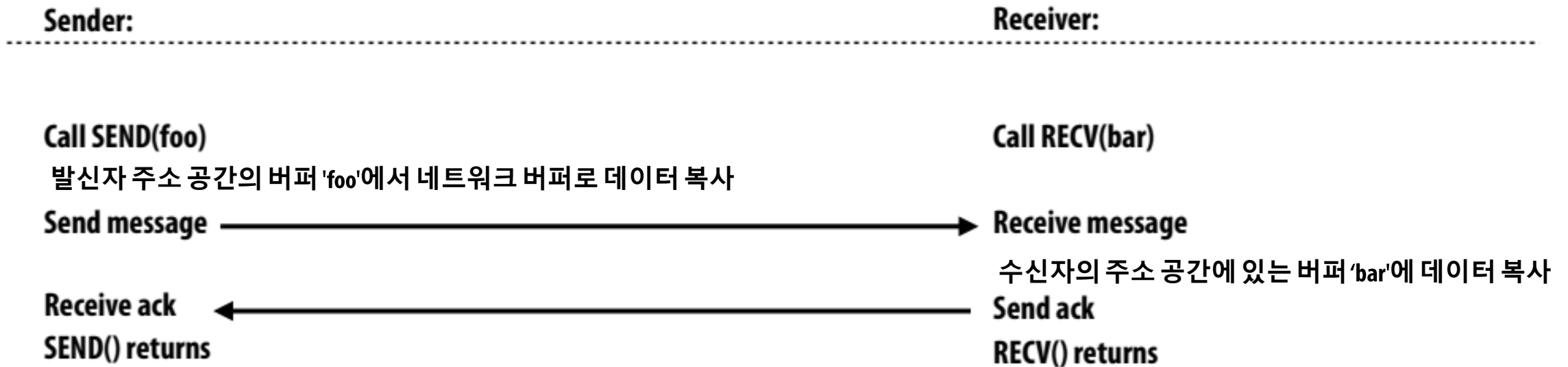
        if (tid != 0) {
            send(&mydiff, sizeof(float), 0, MSG_ID_DIFF);
            recv(&done, sizeof(bool), 0, MSG_ID_DONE);
        } else {
            float remote_diff;
            for (int i=1; i<get_num_threads()-1; i++) {
                recv(&remote_diff, sizeof(float), i, MSG_ID_DIFF);
                my_diff += remote_diff;
            }
            if (my_diff/(N*N) < TOLERANCE)
                done = true;
            for (int i=1; i<get_num_threads()-1; i++)
                send(&done, sizeof(bool), i, MSG_ID_DONE);
        }
    }
}
```

메시지 전달 예제에 대한 참고 사항

- 계산
 - 배열 인덱싱은 로컬 주소 공간에 상대적임.
- 통신:
 - 메시지 송수신으로 수행
 - 대량 전송: 한 번에 전체 행을 통신.
- 동기화:
 - 메시지를 주고받음으로써 수행
 - 메시지를 사용하여 상호 제외, 차단, 플래그를 구현하는 방법을 고려.

Synchronous (blocking) send and receive

- `send()`: 발신자가 수신자의 주소 공간에 메시지 데이터가 있다는 확인을 받으면 호출이 반환됨.
- `recv()`: 수신된 메시지의 데이터가 수신자의 주소 공간에 복사되고 발신자에게 확인을 다시 보내면 호출이 반환



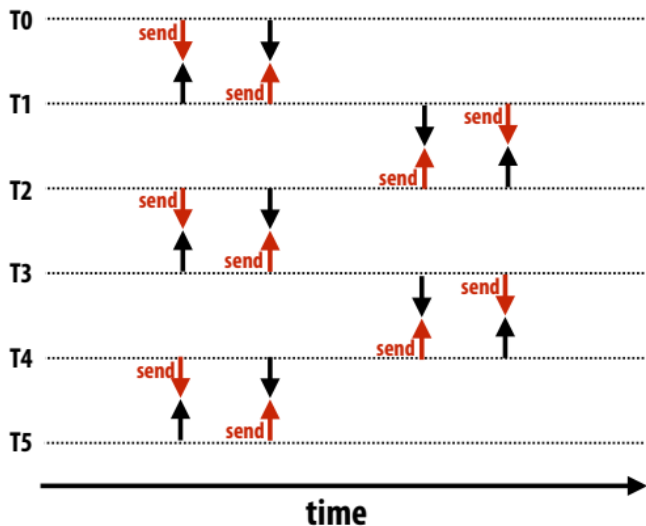
이전 슬라이드에서 구현한 것처럼, 동기식 전송/수신을 사용하는 경우 메시지 전달 솔버에 큰 문제가 있음

Why

이 문제를 어떻게 해결할 수 있나요?
(동기식 전송/수신을 계속 사용하면서)

메시지 전달 솔버(교착 상태를 피하기 위해 수정됨)

“이웃 스레드”에 고스트 행 보내기 및 받기
짝수 스레드 송신 후 수신
홀수 스레드 수신 후 보내기



```
int N;  
int tid = get_thread_id();  
int rows_per_thread = N / get_num_threads();  
  
float* localA = allocate(rows_per_thread+2, N+2);  
  
// assume localA is initialized with starting values  
// assume MSG_ID_ROW, MSG_ID_DONE, MSG_ID_DIFF are constants used as msg ids  
  
////////////////////////////////////  
  
void solve() {  
    bool done = false;  
    while (!done) {  
  
        float my_diff = 0.0f;  
  
        if (tid % 2 == 0) {  
            sendDown(); recvDown();  
            sendUp();  recvUp();  
        } else {  
            recvUp();  sendUp();  
            recvDown(); sendDown();  
        }  
  
        for (int i=1; i<rows_per_thread-1; i++) {  
            for (int j=1; j<n+1; j++) {  
                float prev = localA[i,j];  
                localA[i,j] = 0.2 * (localA[i-1,j] + localA[i,j] + localA[i+1,j] +  
                                     localA[i,j-1] + localA[i,j+1]);  
                my_diff += fabs(localA[i,j] - prev);  
            }  
        }  
  
        if (tid != 0) {  
            send(&mydiff, sizeof(float), 0, MSG_ID_DIFF);  
            recv(&done, sizeof(bool), 0, MSG_ID_DONE);  
        } else {  
            float remote_diff;  
            for (int i=1; i<get_num_threads()-1; i++) {  
                recv(&remote_diff, sizeof(float), i, MSG_ID_DIFF);  
                my_diff += remote_diff;  
            }  
            if (my_diff/(N*N) < TOLERANCE)  
                done = true;  
            if (int i=1; i<gen_num_threads()-1; i++)  
                send(&done, sizeof(bool), i, MSG_ID_DONE);  
        }  
    }  
}
```

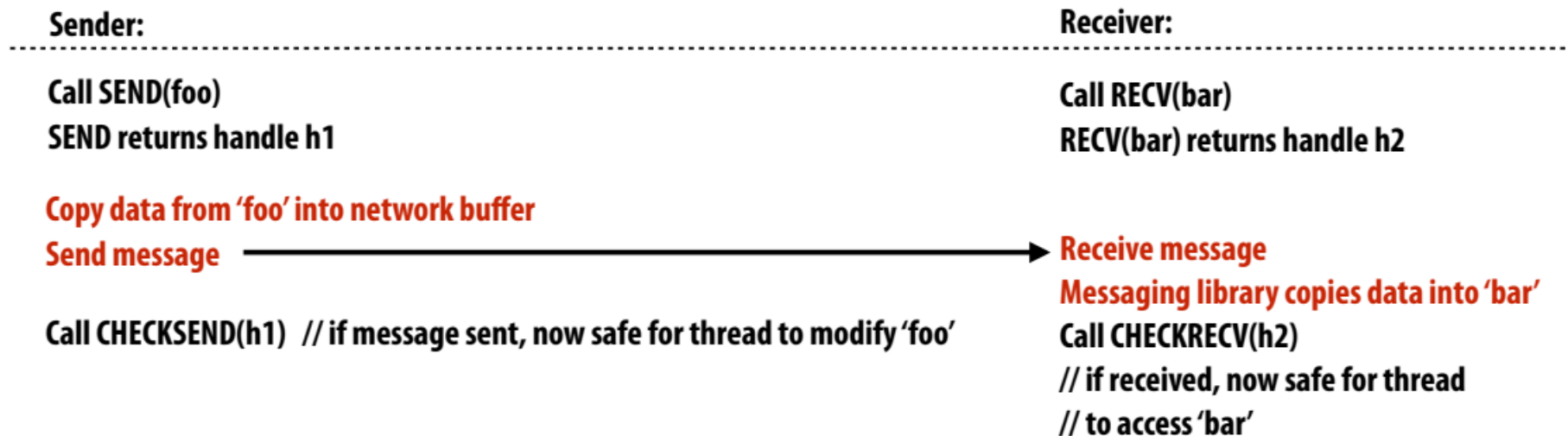
Non-blocking asynchronous send/recv

● send(): 호출 즉시 반환

- 메시지 처리가 스레드 실행과 동시에 발생하므로 send()에 제공된 버퍼는 스레드 호출로 수정할 수 없음
- 스레드 호출은 메시지 전송을 기다리는 동안 다른 작업을 수행할 수 있음

● recv(): 향후 수신 의사를 게시하고 즉시 반환

- 실제 송수신 상태를 확인하려면 checksend(), checkrecv()를 사용.
- 호출 스레드는 메시지 수신을 기다리는 동안 다른 작업을 수행할 수 있음.



RED TEXT = executes concurrently with application thread

커뮤니케이션(communication)에 대해 이야기할 때는 단순히 기계 간의 메시지만을 말하는 것이 아님.
(예: 데이터 센터)

더 많은 예를 들어보면:

칩의 코어 간 통신

코어와 캐시 간의 통신

코어와 메모리 간의 통신

“병렬 시스템을 확장된 메모리 계층 구조”로 생각하자

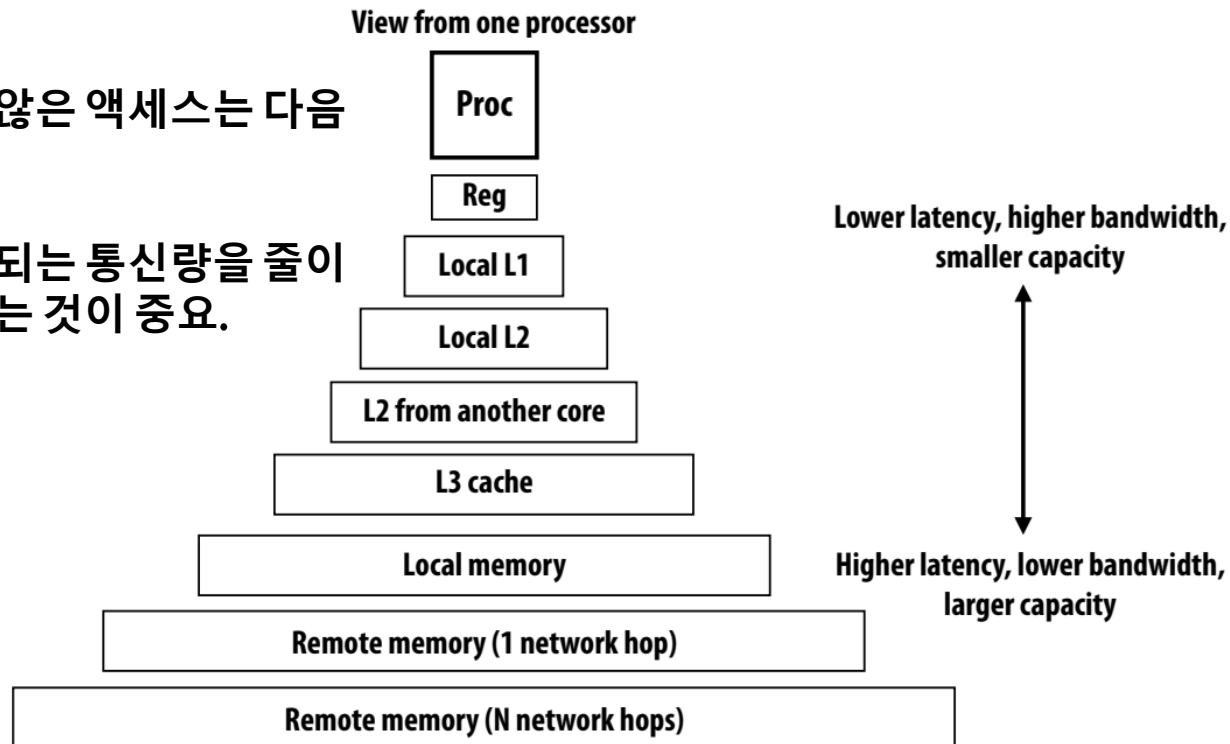
- '통신'을 일반적으로 생각하자:

- 프로세서와 캐시 간의 통신
- 프로세서와 메모리 간의 통신(예: 같은 컴퓨터의 메모리)
- 프로세서와 원격 메모리 간의 통신

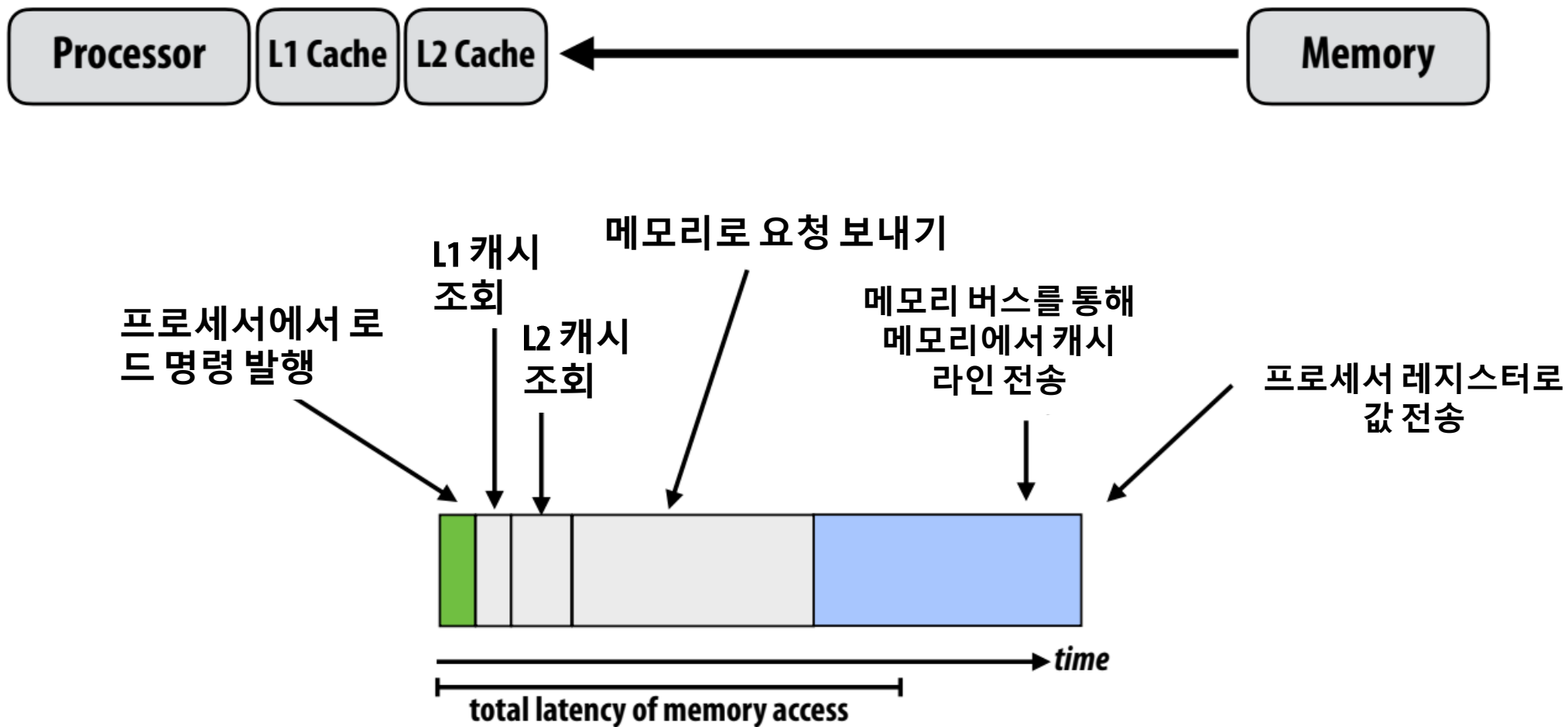
(예: 네트워크 메시지를 전송하여 액세스하는 클러스터의 다른 노드에 있는 메모리)

로컬 메모리에서 충족되지 않은 액세스는 다음 레벨과의 통신을 유발.

따라서 모든 수준에서 수행되는 통신량을 줄이기 위해 로컬리티를 관리하는 것이 중요.

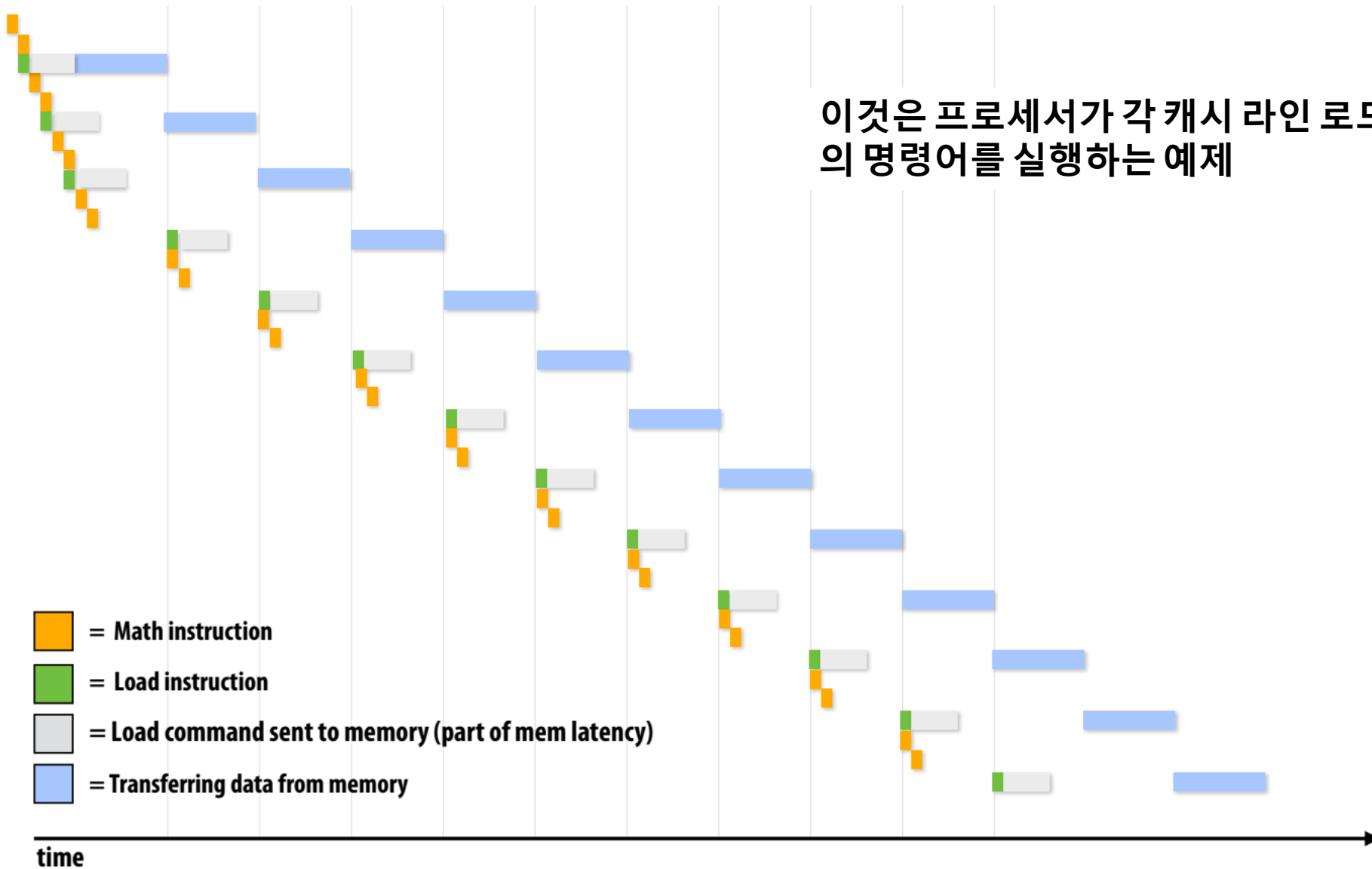


One example: CPU to memory communication

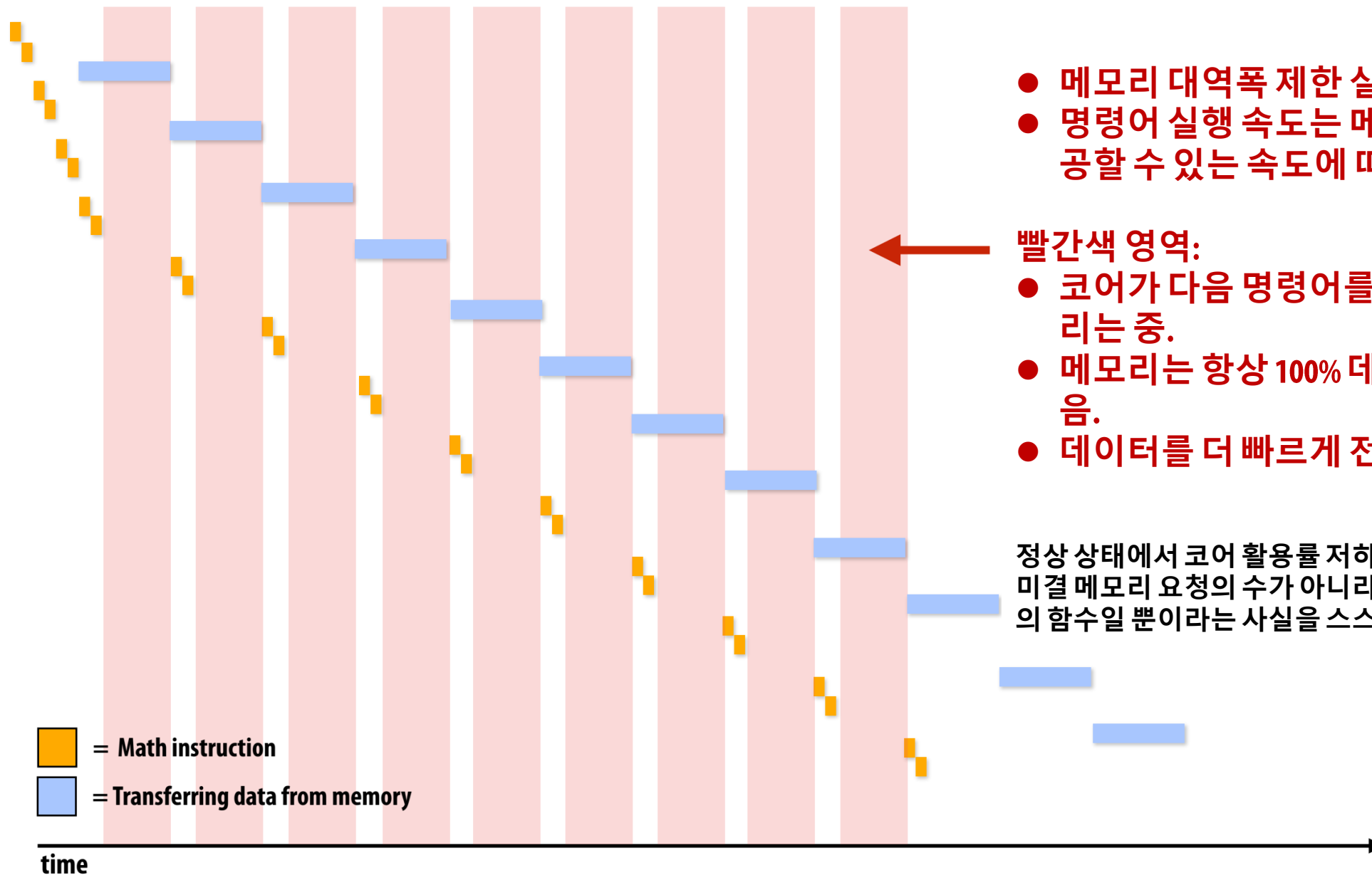


 = Time to send cache line over memory bus

대역폭 제한 실행에 대한 논의를 상기



수학 명령어 완료 속도는 메모리 대역폭에 의해 제한



- 메모리 대역폭 제한 실행!
- 명령어 실행 속도는 메모리가 데이터를 제공할 수 있는 속도에 따라 결정.

빨간색 영역:

- 코어가 다음 명령어를 위해 데이터를 기다리는 중.
- 메모리는 항상 100% 데이터를 전송하고 있음.
- 데이터를 더 빠르게 전송할 수 없음

정상 상태에서 코어 활용률 저하는 메모리 지연 시간이나 미결 메모리 요청의 수가 아니라 명령어와 메모리 처리량의 함수일 뿐이라는 사실을 스스로 확신이 필요함.

Good questions about the previous slide

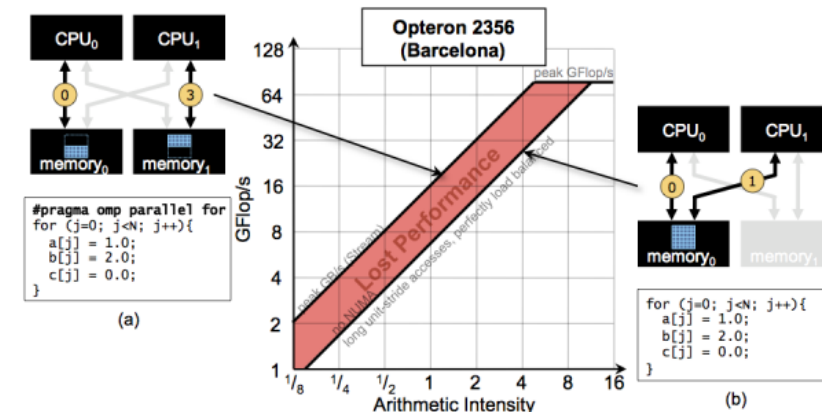
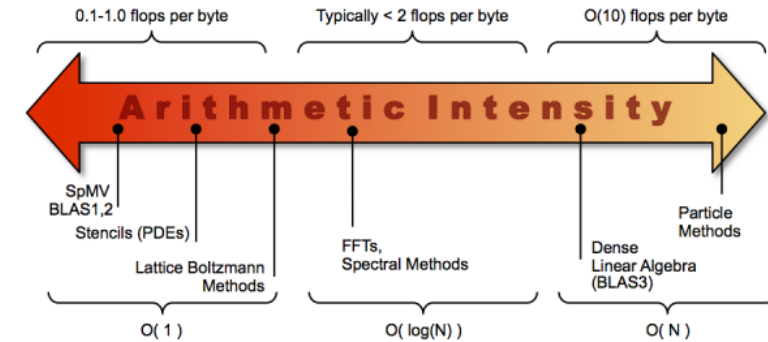
- 메모리 버스가 완전히 활용되고 있다는 것을 그림에서 어떻게 알 수 있나요?
- 메모리 지연 시간이 길어지는 것을 어떻게 설명할 수 있나요(메모리 요청은 파이프라인화 되어 있고 메모리 버스 대역폭은 변경되지 않는다는 점을 염두에 두세요)?
- 메모리 버스 대역폭이 증가하면 수치는 어떻게 달라지나요?
- 수학 명령어와 로드 명령어의 비율이 크게 증가해도 여전히 프로세서가 멈추는 현상이 발생하나요? 왜 그럴까요?

산술 집약도(강도)

계산량(예: 명령어)

통신량 (예: 바이트)

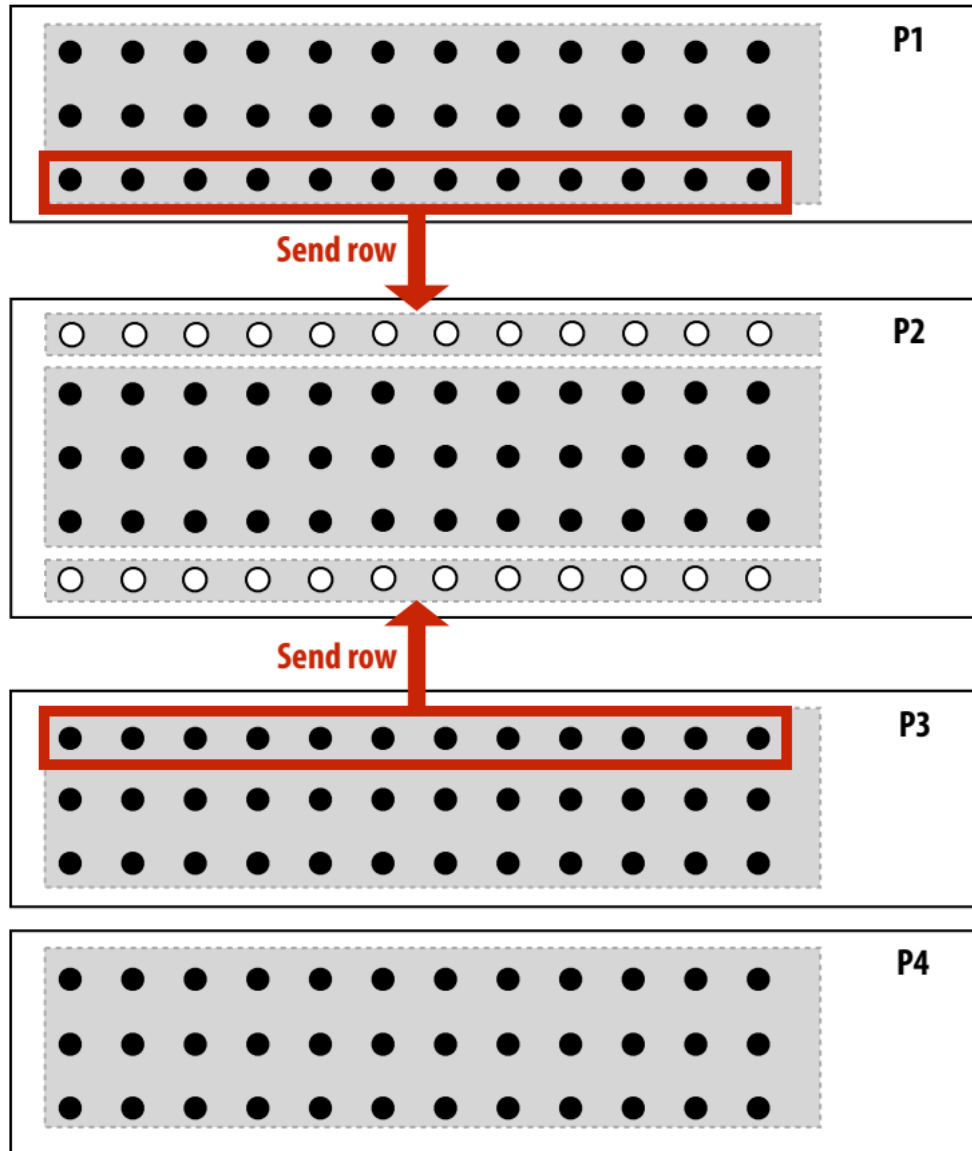
- 분자가 계산 실행 시간인 경우 비율은 코드의 평균 대역폭 요구량을 나타냄.
- 1/“산술 집약도”= 통신 대 계산 비율
 - 어떤 사람들은 통신 대 계산 비율을 언급하는 것을 좋아함.
 - (저자)산술 집약도가 더 직관적인 수치라고 생각하는데, 높을수록 좋기 때문임.
 - 또한 더 멋지게 들림.
- 사용 가능한 대역폭 대비 연산 능력의 비율이 높기 때문에 최신 병렬 프로세서를 효율적으로 활용하려면 높은 산술 집약도(낮은 통신 대 연산 비율)가 필요(요소별 벡터 곱셈 예제 기억).



커뮤니케이션의 두 가지 이유:

내재적인 커뮤니케이션 대 인위적인 커뮤니케이션

Inherent communication(내재적 통신)



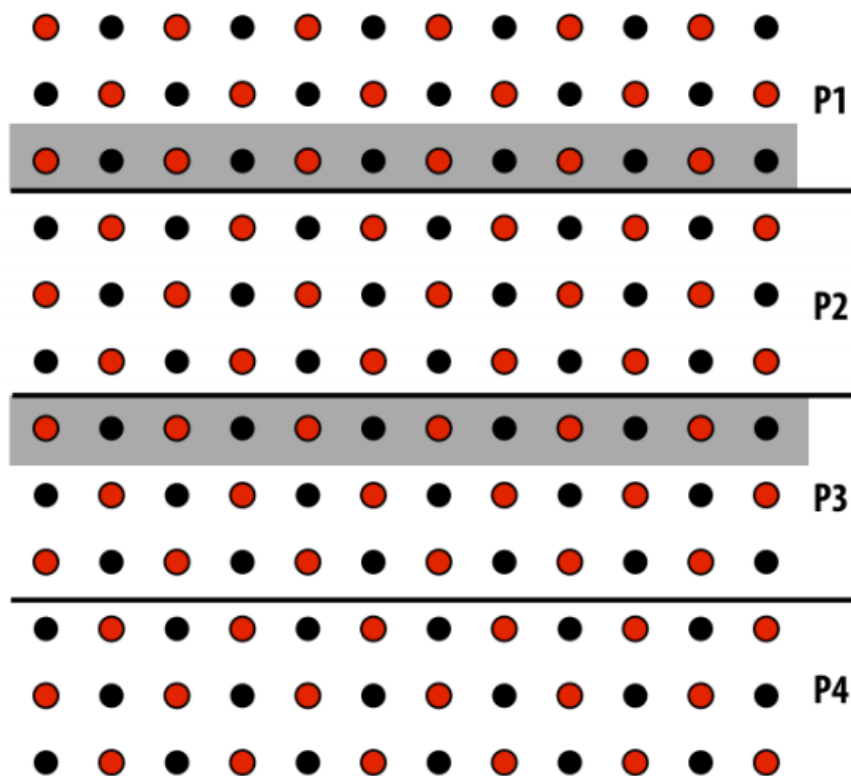
통신은 병렬 알고리즘에서는 반듯이 일어남.
통신은 병렬 알고리즘의 기본.

앞장 메시지 전달 예제에서 고스트 행을 보내는 것은 내재적 통신이었음.

Reducing inherent communication

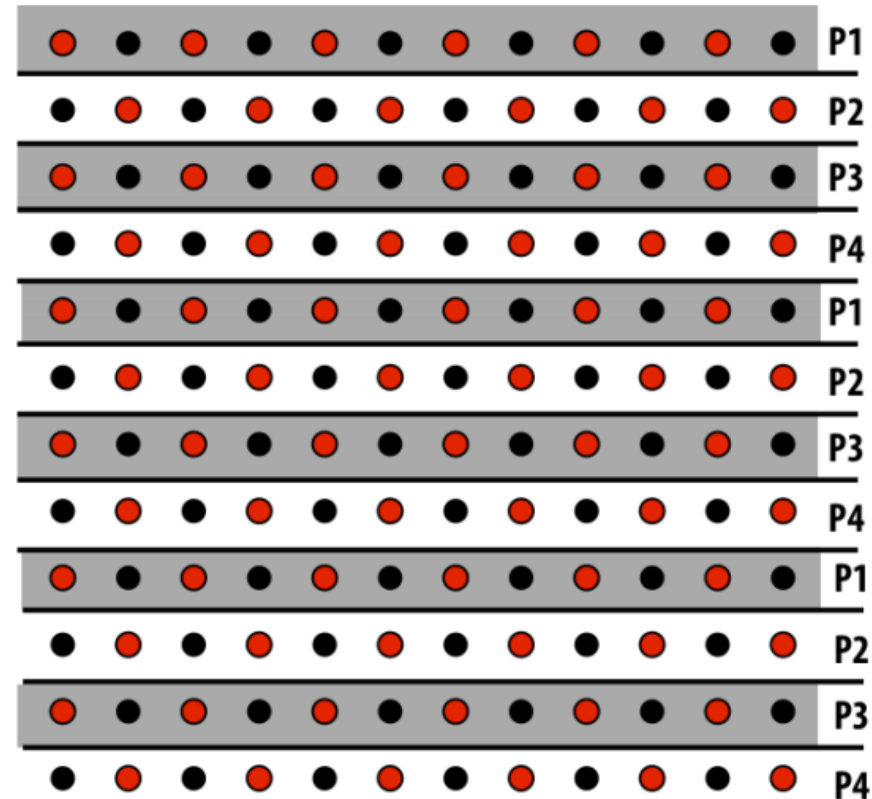
좋은 할당(assignment) 결정은 내재적 커뮤니케이션을 줄일 수 있음.(산술 집약도(강도) 증가)

1D blocked assignment: N x N grid



$$\frac{\text{elements computed (per processor)} \approx N^2/P}{\text{elements communicated (per processor)} \approx 2N} \propto N/P$$

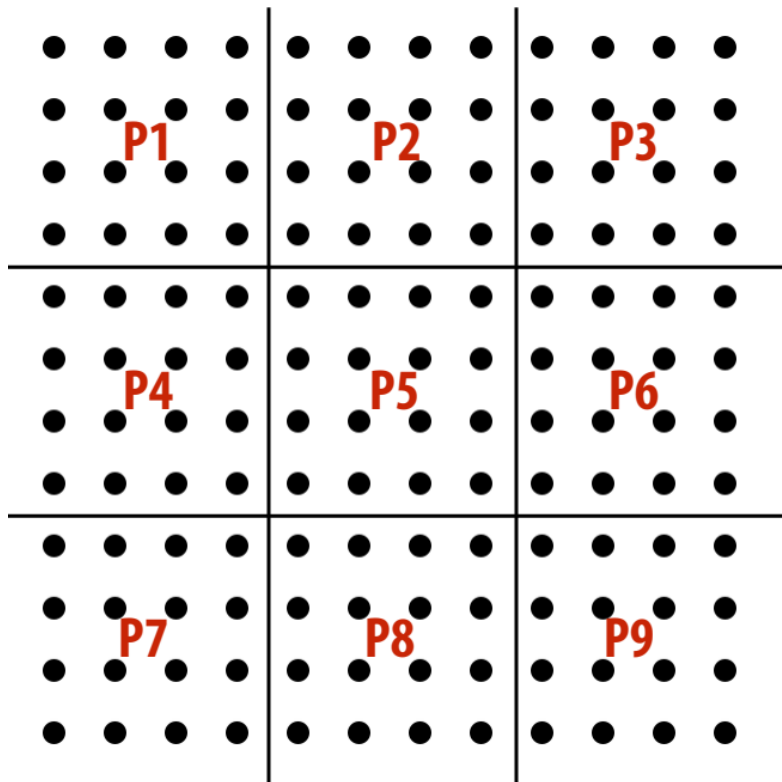
1D interleaved assignment: N x N grid



$$\frac{\text{elements computed}}{\text{elements communicated}} = 1/2$$

Reducing inherent communication

2D blocked assignment: $N \times N$ grid



N^2 elements

P processors

elements computed:
(per processor)

$$\frac{N^2}{P}$$

elements communicated:
(per processor)

$$\propto \frac{N}{\sqrt{P}}$$

arithmetic intensity:

$$\frac{N}{\sqrt{P}}$$

- 1D block 할당보다 점근적으로 우수한 통신 확장성
- 통신 비용은 P 에 따라 준선형적(Sub-Linear)으로 증가.
- 할당은 알고리즘의 2D 로컬리티를 캡처함.

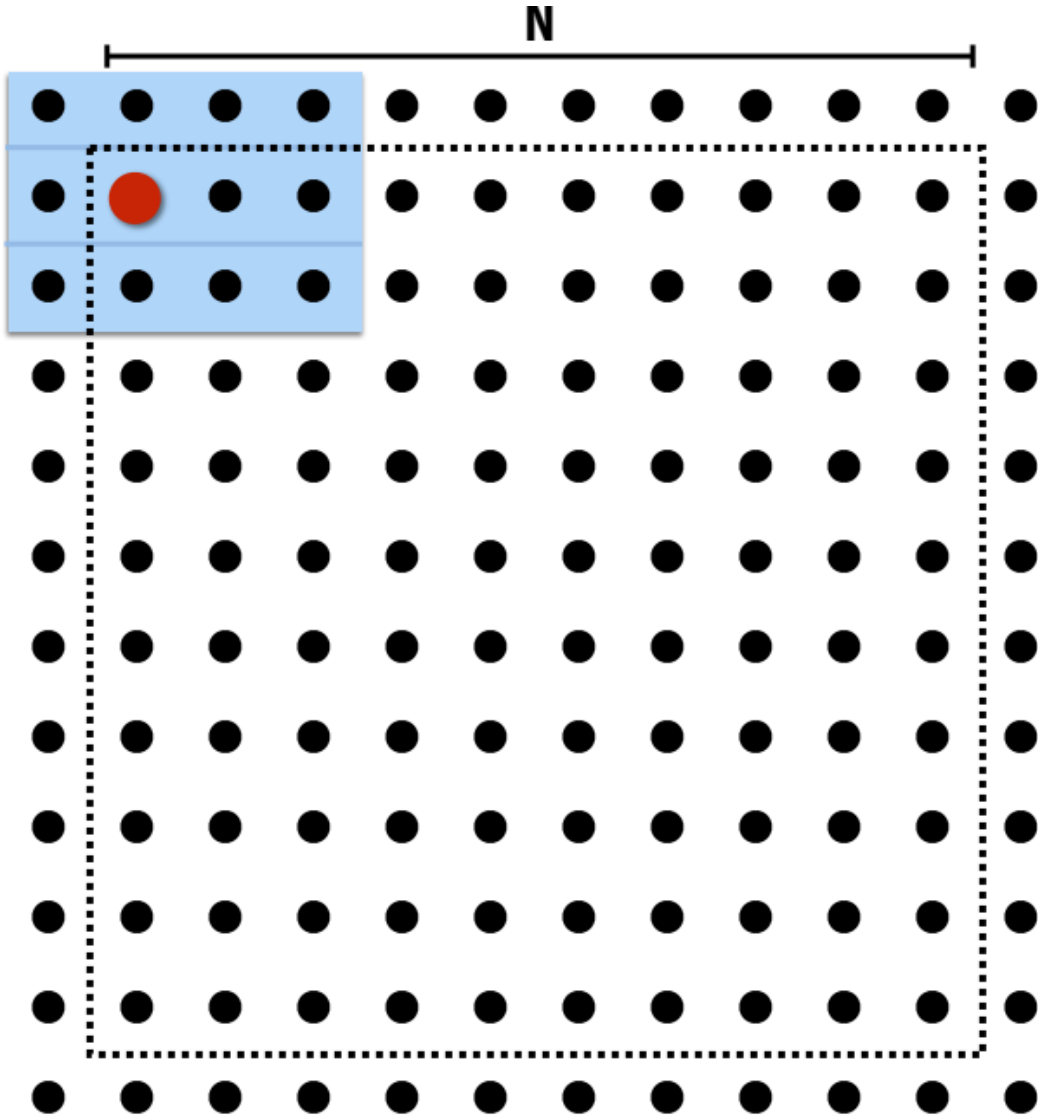
Artifactual communication

- **내재적 통신(Inherent communication):** 지정된 할당이 주어진 알고리즘을 수행하기 위해 기본적으로 프로세서 간에 이동해야 하는 정보(무제한 용량 캐시, 최소 세분성 전송(minimum granularity transfers) 등을 가정)
- **Artifactual communication:** 기타 모든 통신(Artifactual 통신은 시스템 구현의 실질적인 세부 사항에서 비롯됨)

Example:
Artifactual communication는
캐시의 동작(behavior)에서 발생

이 경우: 통신은 메모리와 프로세서 간에 이루어집니다.

그리드 솔버의 데이터 액세스: 행(Row)우선 탐색



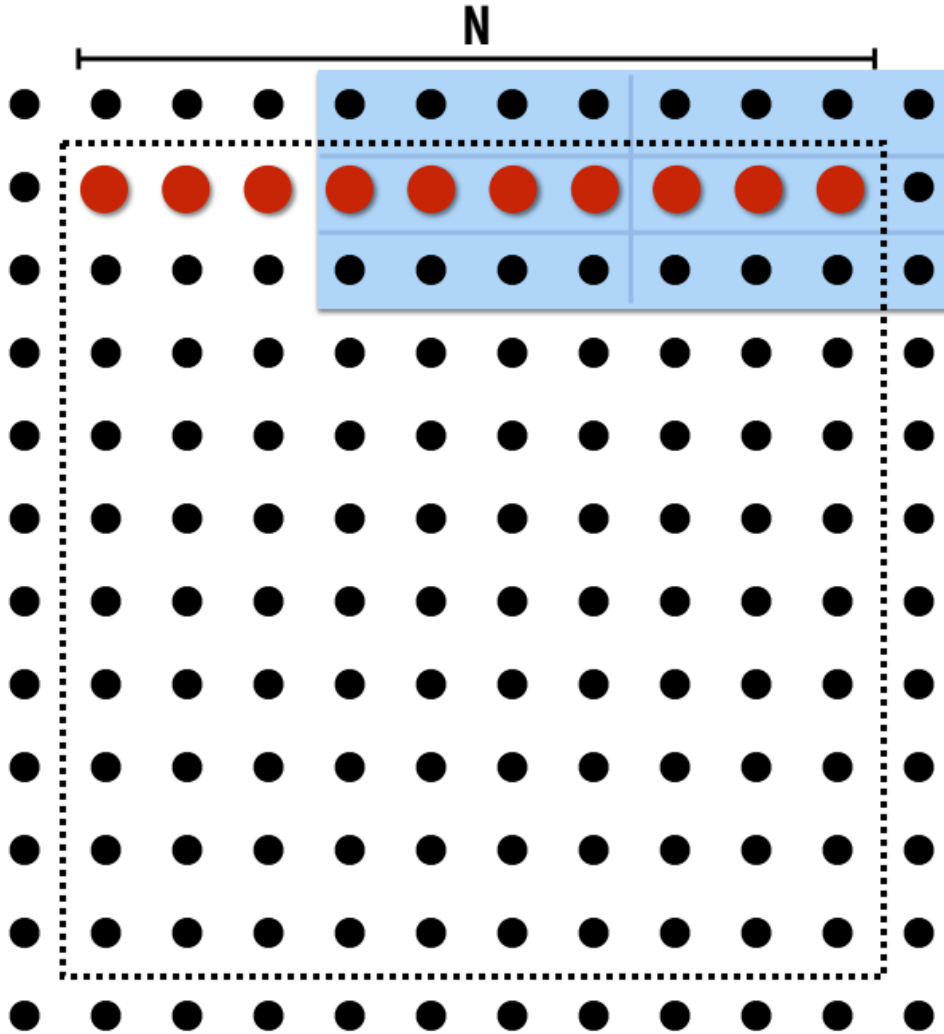
행(row)우선 격자 레이아웃을 가정.

캐시 라인이 4개의 그리드 요소라고 가정.

캐시 용량은 24개의 그리드 요소(6줄)임.

그리드 솔버 애플리케이션에서 데이터 액세스를 리콜.
파란색 요소는 빨간색 요소로 업데이트가 완료된 후 캐시
에 있는 데이터를 나타냄.

그리드 솔버의 데이터 액세스: 행(Row)우선 탐색



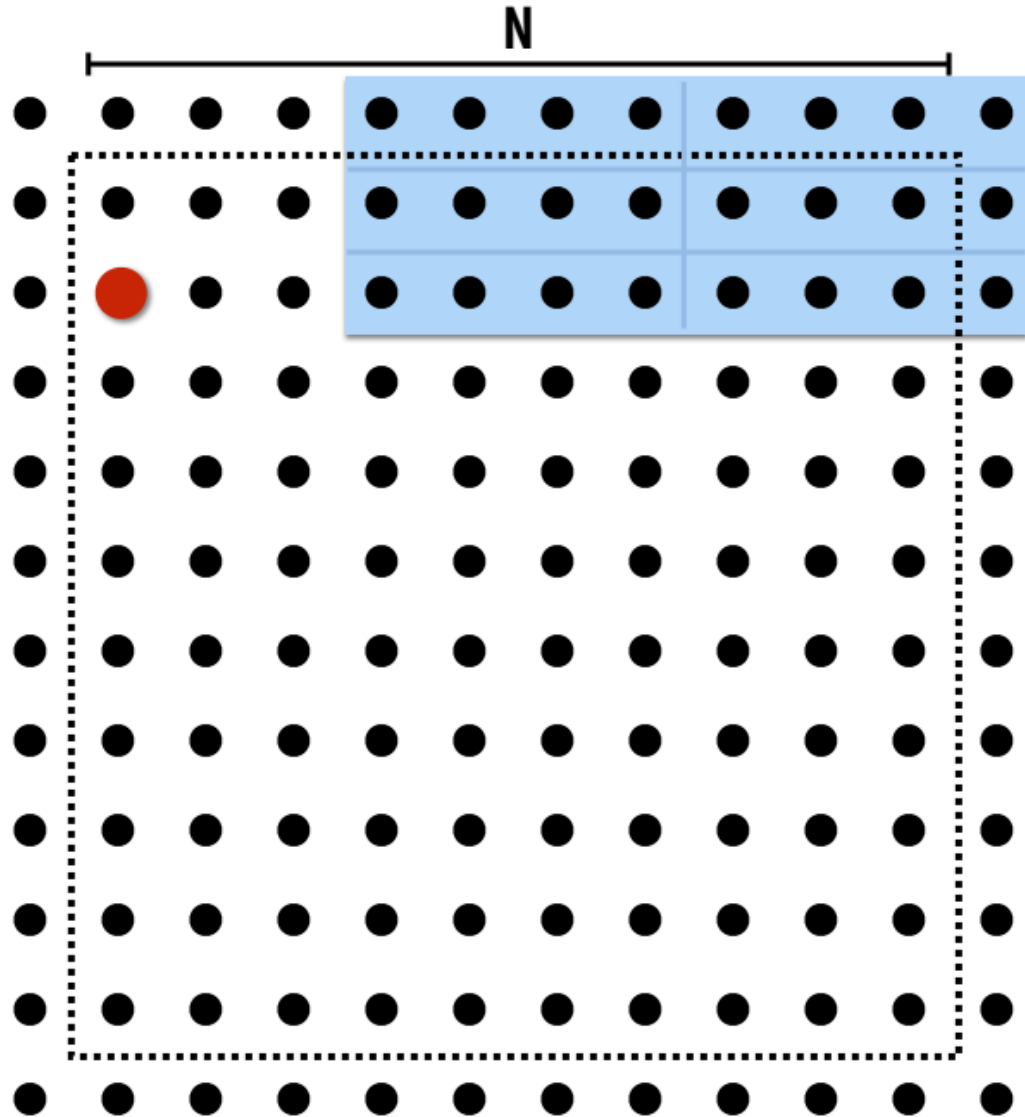
행(row)우선 격자 레이아웃을 가정.

캐시 라인이 4개의 그리드 요소라고 가정.

캐시 용량은 24개의 그리드 요소(6줄)임.

파란색 요소는 첫 번째 행 처리가 끝날 때 캐시에 있는 데이터를 나타냄.

행(Row)우선 탐색 문제: 동일한 데이터에 액세스하는 데 시간이 오래 걸림



행(row)우선 격자 레이아웃을 가정.

캐시 라인이 4개의 그리드 요소라고 가정.

캐시 용량은 24개의 그리드 요소(6줄)임.

$(x,y)=(0,1), (1,1), (2,1), (0,2), (2,2)$ 요소는 이전에 액세스되었지만 2행의 첫 번째 출력 요소를 처리하기 시작할 때 더 이상 캐시에 존재하지 않습니다.

결과적으로 이 프로그램은 출력 요소 4개당 3개의 캐시 라인을 로드함.

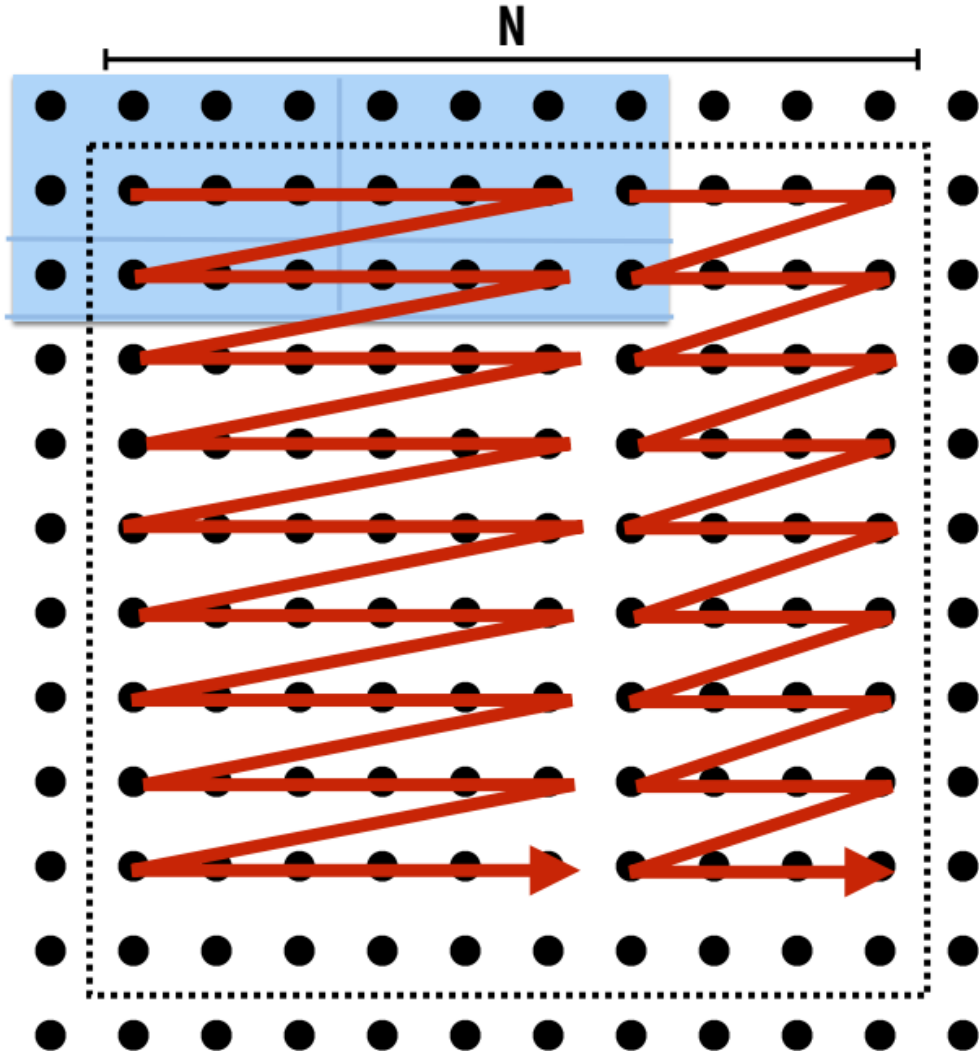
Artifactual communication examples

- 시스템은 데이터 전송의 최소 단위(granularity)를 가짐 (시스템은 애플리케이션에 필요한 것보다 더 많은 데이터를 통신해야 함).
 - 프로그램은 4바이트 부동 소수점 값 하나를 로드하지만 64바이트 캐시 라인 전체를 메모리에서 전송해야 함(필요한 것보다 16배 더 많은 통신).
- 시스템 작동 시 불필요한 통신이 발생할 수 있음:
 - 프로그램이 16개의 연속된 4바이트 부동 소수점 값을 저장하고, 그 결과 64바이트 캐시 라인 전체를 메모리에서 로드하여 완전히 덮어쓰는 후 다시 메모리에 저장 (캐시 라인 전체를 덮어썼으므로 2배의 오버헤드... 불필요한 로드)
- 유한한 복제 용량: 캐시가 너무 작아 액세스 사이에 데이터를 유지할 수 없기 때문에 동일한 데이터가 프로세서에 여러 번 통신됨(capacity misses)

Techniques for reducing communication

그리드 탐색 순서를 변경하여 시간적 로컬리티 향상

“차단”: 용량 누락(capacity misses)을 줄이기 위한 재주문 계산



행(row)우선 격자 레이아웃을 가정.

캐시 라인이 4개의 그리드 요소라고 가정.

캐시 용량은 24개의 그리드 요소(6줄)임.

“Blocked” iteration order

(다이어그램은 첫 번째 블록의 첫 번째 행부터 작업을 완료한 후 캐시 상태를 보여줌.)

이제 출력 요소 6개마다 캐시 라인 2개를 로드함.

Optimization: improve arithmetic intensity by sharing data

```
void add(int n, float* A, float* B, float* C) {  
    for (int i=0; i<n; i++)  
        C[i] = A[i] + B[i];  
}
```

← 두 개의 로드, 수학 연산당 하나의 저장소
(arithmetic intensity = 1/3)

```
void mul(int n, float* A, float* B, float* C) {  
    for (int i=0; i<n; i++)  
        C[i] = A[i] * B[i];  
}
```

← 두 개의 로드, 수학 연산당 하나의 저장소
(arithmetic intensity = 1/3)

```
float* A, *B, *C, *D, *E, *tmp1, *tmp2;
```

```
// assume arrays are allocated here
```

```
// compute E = D + ((A + B) * C)
```

```
add(n, A, B, tmp1);
```

```
mul(n, tmp1, C, tmp2);
```

```
add(n, tmp2, D, E);
```

← Overall arithmetic intensity = 1/3

```
void fused(int n, float* A, float* B, float* C, float* D, float* E) {  
    for (int i=0; i<n; i++)  
        E[i] = D[i] + (A[i] + B[i]) * C[i];  
}
```

← 4개의 로드, 3개 수학 연산당 하나의 저장소
(arithmetic intensity = 3/5)

```
// compute E = D + (A + B) * C
```

```
fused(n, A, B, C, D, E);
```

상단의 코드가 더 모듈화되어 있음(예: Python의 numPy와 같은 배열 기반 수학 라이브러리).
아래쪽의 코드가 훨씬 더 성능이 좋다. 왜 그럴까요?

최적화: 데이터 공유를 통한 산술 강도 향상

- **익스플로잇 공유(Exploit sharing): 동일한 데이터에서 작동하는 작업의 공동 배치**

- 동일한 프로세서에서 동일한 데이터 구조에서 동시에 작업하는 스레드를 예약
- 고유한 통신 감소

Contention(경합)

Contention(논쟁)

예: 근무 시간: 오후 3시~3시 20분(예약 없음)

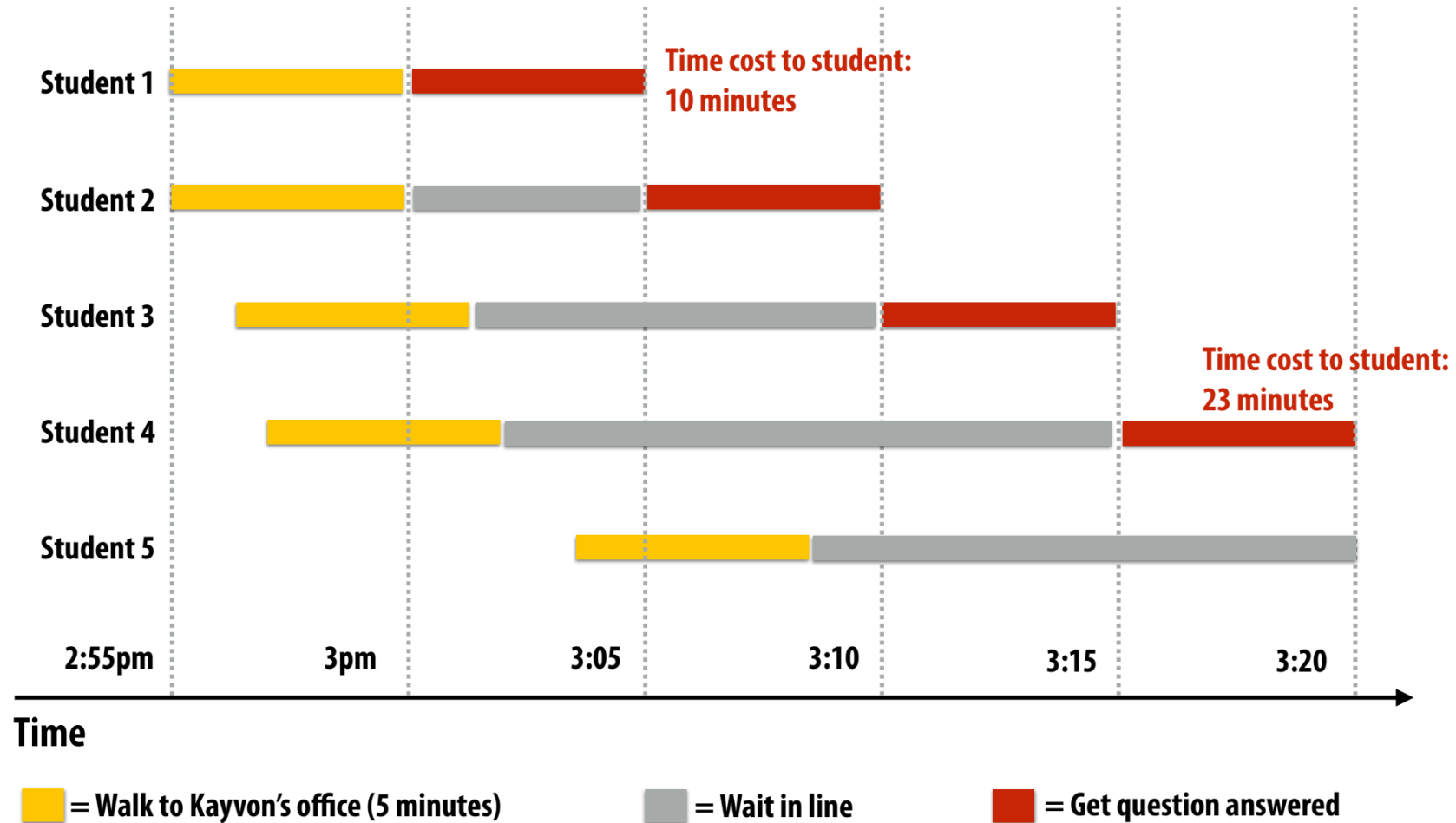
수행해야 할 작업: 케이본 교수가 학생의 질문을 도와줌

실행 리소스: 케이본 교수

운영 단계

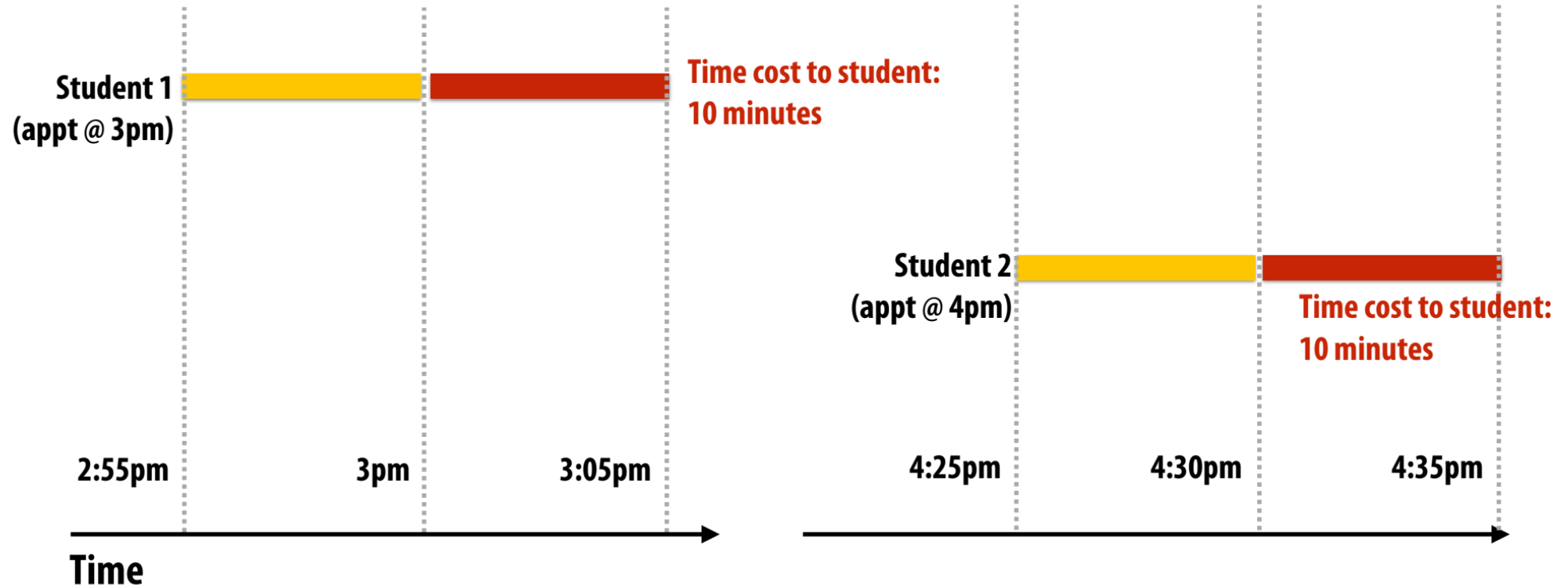
1. 학생이 바이트 카페에서 케이본 교수의 사무실까지 걸어갑니다(5분).
2. 학생이 줄을 서서 기다립니다(필요한 경우).
3. 학생은 통찰력 있는 답변을 통해 질문에 대한 답을 얻습니다 (5분).

Example: office hours from 3-3:20pm (no appointments)



문제: 공유 리소스에 대한 **경합으로 인해** 전체 운영 시간이 길어지고 학생에게 더 많은 비용이 발생할 수 있음

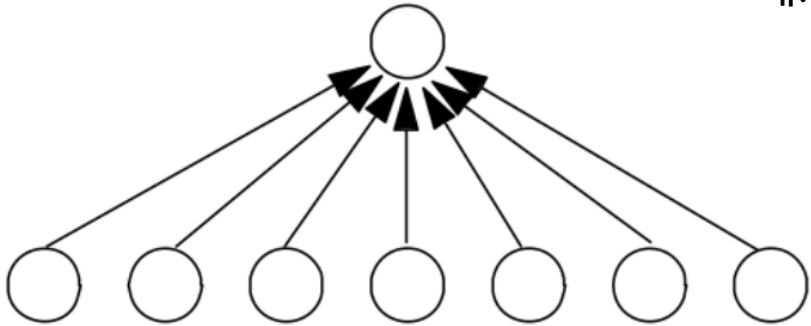
예: 두 명의 학생이 오후 3시와 오후 4시 30분에 코스 자료에 대해 저와 상담하기로 약속을 잡았음.



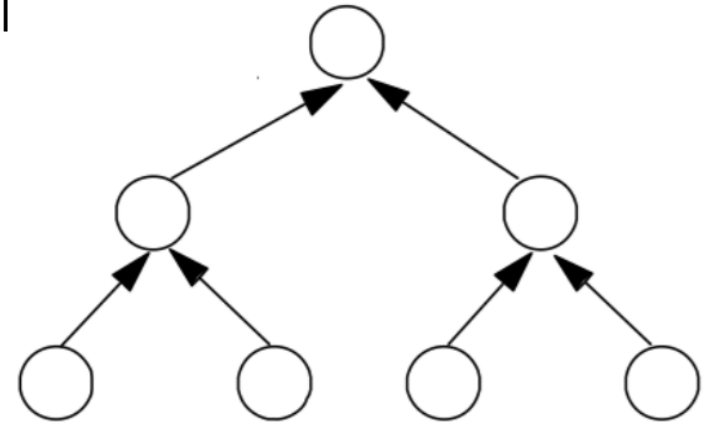
● Contention

- 리소스는 주어진 처리량(단위 시간당 트랜잭션 수)으로 작업을 수행할 수 있음.
 - 메모리, 통신 링크, 서버, 근무 시간 중 CA 등
- 짧은 시간 내에 리소스에 대한 많은 요청이 이루어질 때 경합이 발생함.
(리소스는 “핫스팟”)

예: 공유 변수 업데이트하기



Flat communication:
높은 경합 가능성
(하지만 경합이 없는 경우 지연 시간이 짧음)



Tree structured communication:
경합 감소
(하지만 경합이 없는 상태에서는 지연 시간이 더 길어짐)

Example: distributed work queues reduce contention

(단일 공유 작업 대기열에 대한 액세스 권한 경합)

Subproblems

(a.k.a. "tasks", "work to do")

Set of work queues

(일반적으로 작업자 스레드당 하나)

Worker threads:

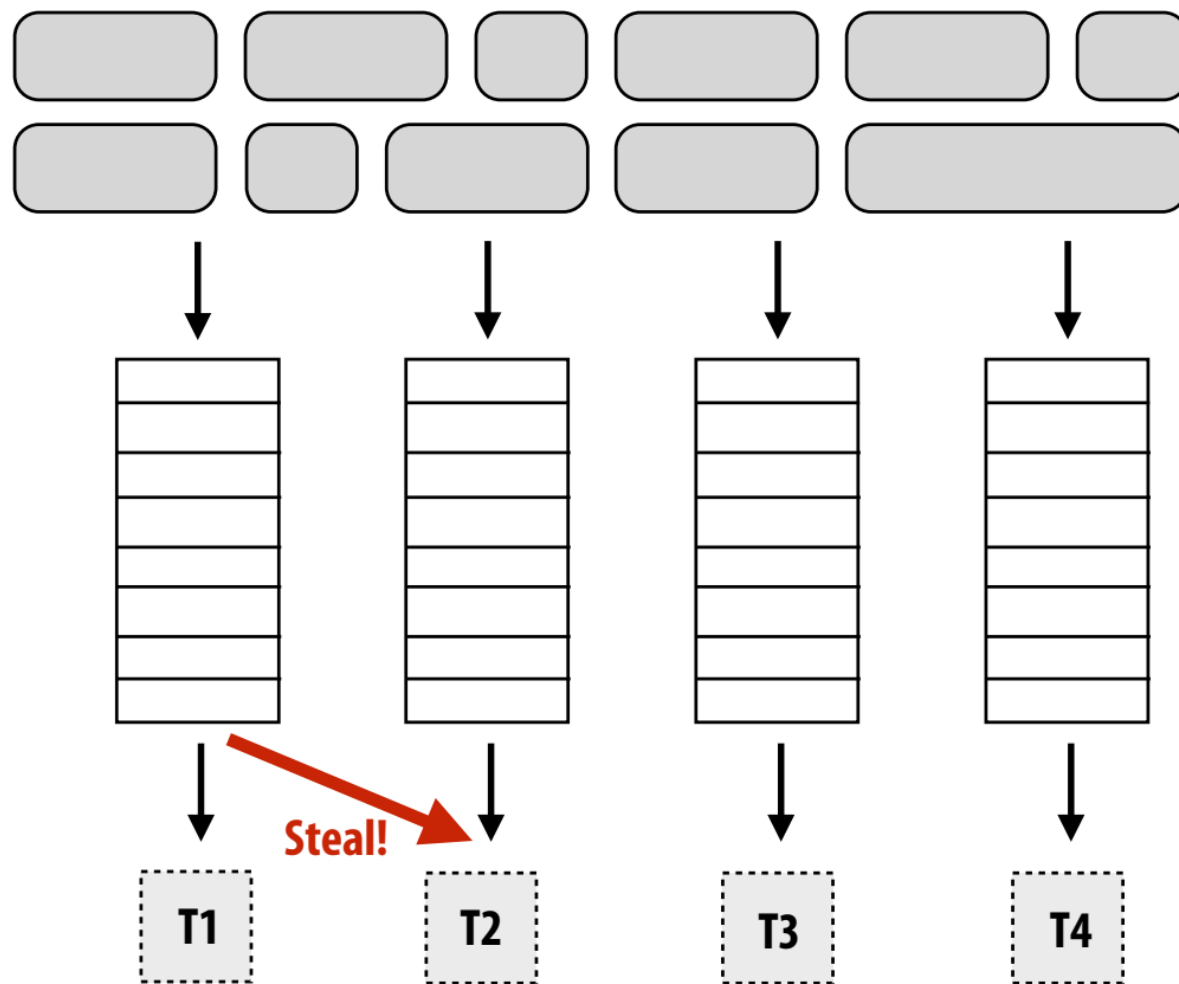
자체 작업 대기열(OWN work queue)에서 데이터 가져오기
새 작업을 자체 작업 대기열로 푸시함.

(모든 프로세서에 할 일이 있을 때 경합 없음)

로컬 작업 큐가 비어 있는 경우...

임의의 작업 대기열에서 작업 훔치기

(어차피 스레드가 유휴 상태였을 것이므로
이 시점에서는 동기화가 정상.)



Summary: reducing communication costs

▪ 발신자/수신자에 대한 통신 오버헤드 감소

- 더 적은 수의 메시지를 보내고, 메시지를 더 크게 만듦(오버헤드 상각).
- 많은 작은 메시지를 큰 메시지로 통합

▪ 통신 지연 시간 단축

- 애플리케이션 작성자: 로컬리티를 활용하도록 코드 재구성
- 하드웨어 구현자: 통신 아키텍처 개선

▪ 경합 감소

- 경합 중인 리소스 복제(예: 로컬 복사본, 세분화된 잠금, local copies, fine-grained locks)
- 경합 중인 리소스에 대한 액세스 시간 분산

▪ 통신/계산 중첩 증가

- 애플리케이션 작성자: 비동기 통신 사용(예: 비동기 메시지)
- HW 구현자: 파이프라이닝, 멀티스레딩, 프리페칭, 아웃오더 실행
- 애플리케이션에 추가 동시성 필요(실행 단위 수보다 더 많은 동시성)

병렬 소프트웨어의 성능을 이해하기 위한 몇 가지 요령

기억하세요:

항상, 항상, 항상 가장 간단한 병렬 솔루션을 먼저 시도한 다음
성능을 측정하여 현재 상태를 확인하세요.

A useful performance analysis strategy

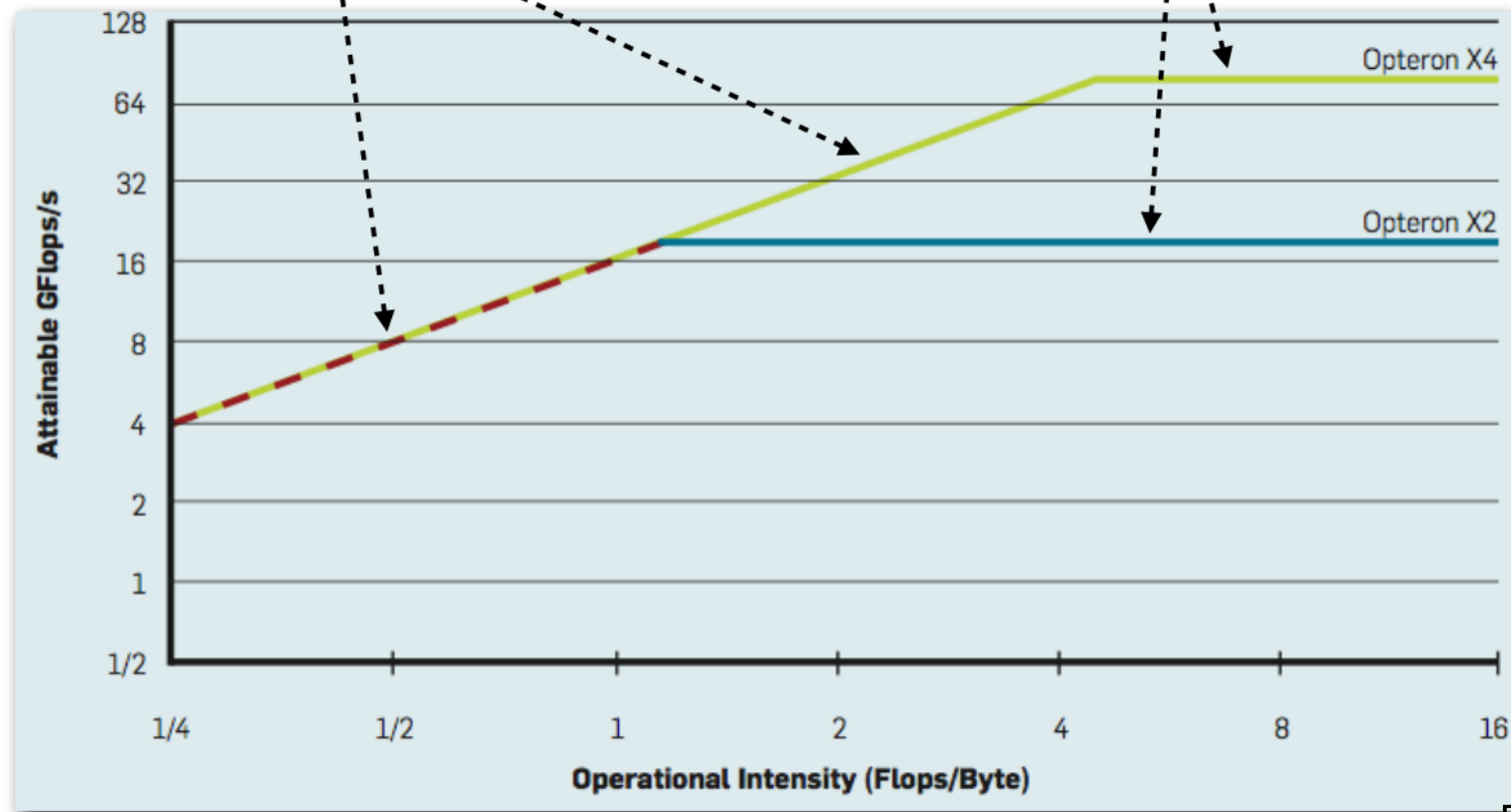
- 연산, 메모리 대역폭(또는 메모리 지연 시간) 또는 동기화에 의해 성능이 제한되는지 확인?
- “높은 워터마크”를 설정해 보세요.
 - 실제로 할 수 있는 최선은 무엇인가요?
 - 구현이 최상의 시나리오에 얼마나 근접해 있나요?

Roofline model

- 아래 그림에서 x축의 다른 점은 서로 다른 연산 강도를 가진 서로 다른 프로그램에 해당합니다.
- y축은 주어진 연산 강도를 가진 프로그램에서 얻을 수 있는 최대 명령어 처리량입니다.

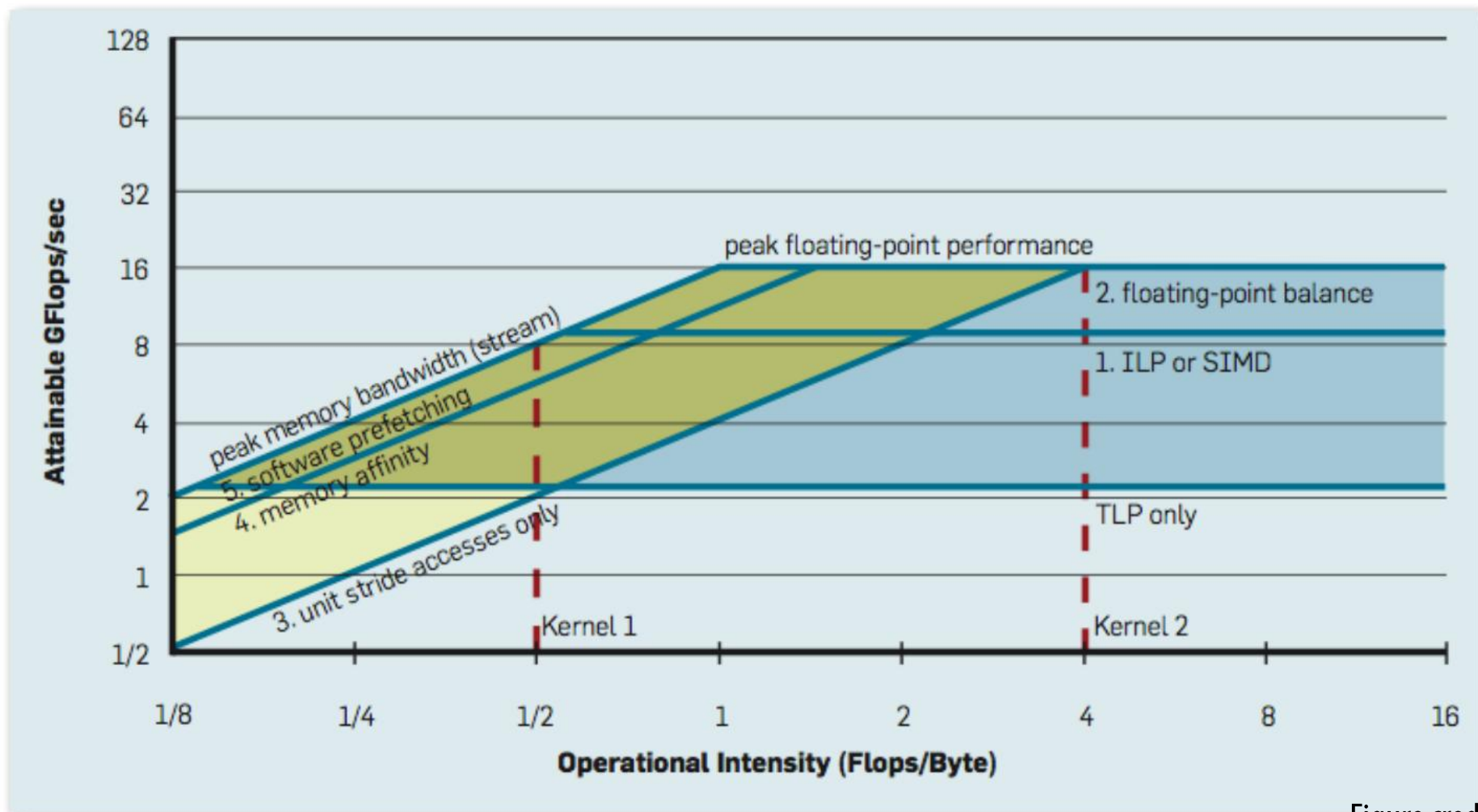
대각선 영역: 메모리 대역폭 제한 실행

수평 영역: 제한된 실행 계산



Roofline model: optimization regions

- 벤치마크에서 다양한 수준의 최적화 사용
- (예: SIMD 명령어를 사용했을 때와 사용하지 않았을 때의 최고 성능)



높은 워터마크 설정 *

- “수학”(비메모리 명령어) 추가하기
 - 연산이 추가되면 실행 시간이 연산 수에 따라 선형적으로 증가하나요?
(그렇다면 코드가 명령어 속도에 제한이 있다는 증거.)
 - 거의 모든 math를 제거하되, 동일한 데이터 로드
 - 실행 시간이 얼마나 단축될까요? 많지 않다면 메모리 병목 현상을 의심할 수 있음.
 - 모든 배열 액세스를 $A[0]$ 으로 변경함.
 - 코드가 얼마나 빨라질까요?
(데이터 액세스의 로컬리티 개선에 따른 이점의 상한선을 설정함.)
 - 모든 원자 연산 또는 잠금을 제거함.
 - 코드가 얼마나 빨라지나요?(여전히 거의 동일한 양의 작업을 수행한다면)
(동기화 오버헤드 감소로 인한 이점에 대한 상한선을 설정함.)
- * 계산, 메모리 액세스, 동기화는 거의 완벽하게 겹치지 않음. 따라서 전체 성능은 컴퓨팅이나 대역폭 또는 동기화에 의해 전적으로 결정되는 경우는 거의 없음.
 - 그럼에도 불구하고 위의 프로그램 수정에 대한 성능 변화의 민감도는 지배적인 비용을 나타내는 좋은 지표가 될 수 있음.

Use profilers/performance monitoring tools

- 왼쪽 이미지는 Chrome에서 웹을 탐색하는 동안 OS X의 활동 모니터에 표시된 'CPU 사용량'(쿼드코어 Core i7 CPU가 탑재된 노트북에서).
 - 그래프는 OS가 프로세서 실행 컨텍스트에 프로세스 스레드를 예약한 시간의 비율을 표시합니다.
 - 성능 최적화에 큰 도움이 되지 않음
- 모든 최신 프로세서에는 낮은 수준의 이벤트 “성능 카운터(performance counters)”가 있음
 - 완료된 명령어, 클럭 틱, L2/L3 캐시 히트/미스, 메모리 컨트롤러에서 읽은 바이트 등과 같은 중요한 세부 정보를 계산하는 레지스터임.
- 예시: 인텔의 성능 카운터 모니터 도구는 이러한 레지스터에 액세스하기 위한 C++ API를 제공.

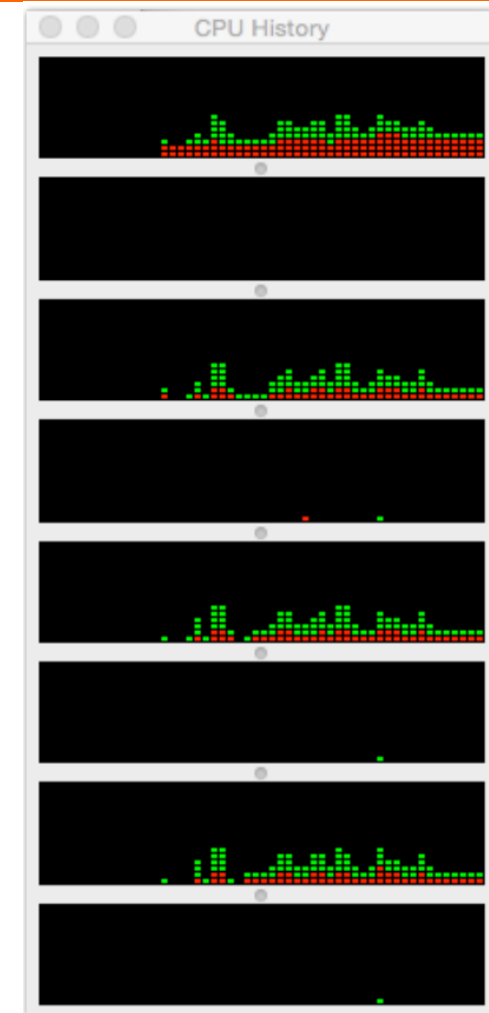
```
PCM *m = PCM::getInstance();
SystemCounterState begin = getSystemCounterState();

// code to analyze goes here

SystemCounterState end = getSystemCounterState();

printf("Instructions per clock: %f\n", getIPC(begin, end));
printf("L3 cache hit ratio: %f\n", getL3CacheHitRatio(begin, end));
printf("Bytes read: %d\n", getBytesReadFromMC(begin, end));
```

- Also see Intel VTune, PAPI, oprofile, etc



Bonus slides:

**문제 크기 문제를 이해하면 프로그램 성능
을 평가할 때 매우 유용합니다.**

- 여러분은 [여기에 좋아하는 칩 회사를 입력]에 고용되었습니다.
- 첫날 출근하자마자 상사가 이렇게 말합니다.
- “수석 아키텍트들이 모두 올해를 쉬기로 결정했습니다. 당신의 임무는 다음 병렬 프로세서의 설계를 주도하는 것입니다.”라고 말합니다.
- 어떤 질문을 할 수 있을까요?

상사가 회사에 가장 중요한 애플리케이션을 선택하면서 “이 애플리케이션에서 좋은 성과를 보여주길 바란다”고 말합니다.

- 절대적인 성과?

- 흔히 벽시계 시간으로 측정
- 다른 예: 초당 작업 수

- 속도 향상: 병렬 처리로 인한 성능 향상?

- 순차적 프로그램의 실행 시간 / p 프로세서에서의 실행 시간
- p 프로세서에서의 초당 작업 수 / 순차적 프로그램의 초당 작업 수

- 효율성?

- 단위 리소스당 성능
- 예: 칩 면적당, 달러당, 와트당 초당 작업 수

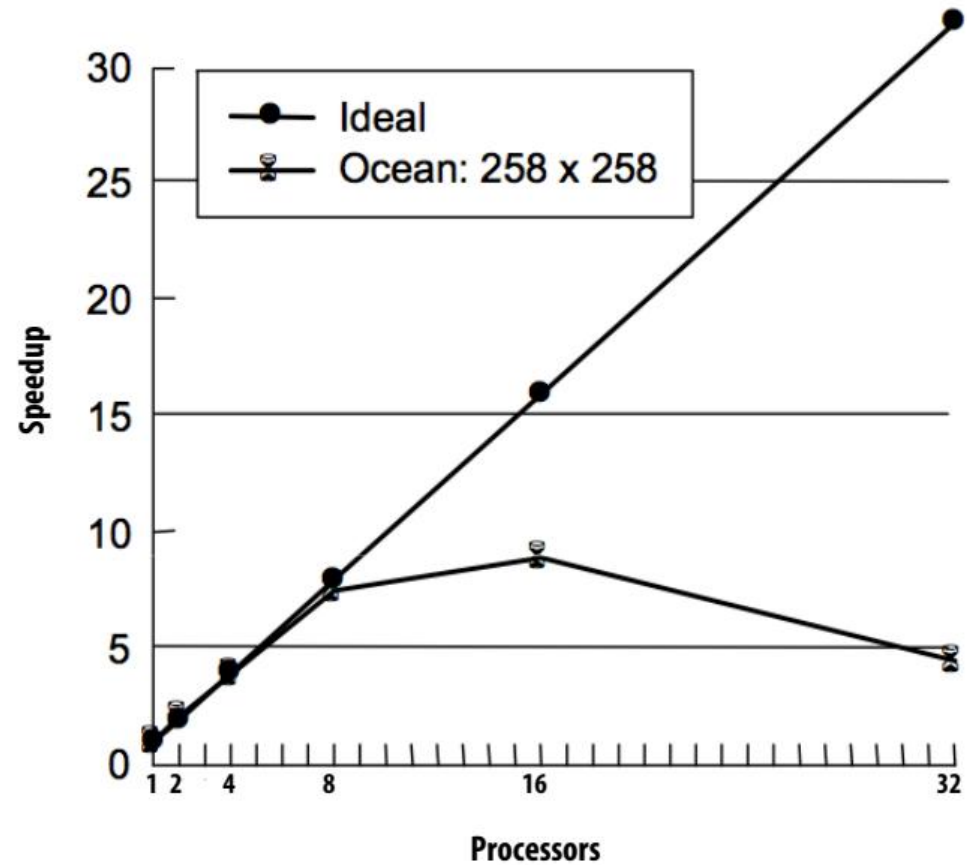
스케일링 측정

- 그리드 솔버 예시를 살펴보겠습니다.
 - 병렬 처리를 허용하도록 알고리즘을 변경했습니다.
 - 새 알고리즘은 더 느리게 수렴되어 솔버의 더 많은 반복이 필요할 수 있습니다.
- 하나의 프로세서에서 실행되는 프로그램의 병렬 버전 또는 최상의 순차 프로그램 성능을 기준으로 속도 향상을 측정해야 할까요?

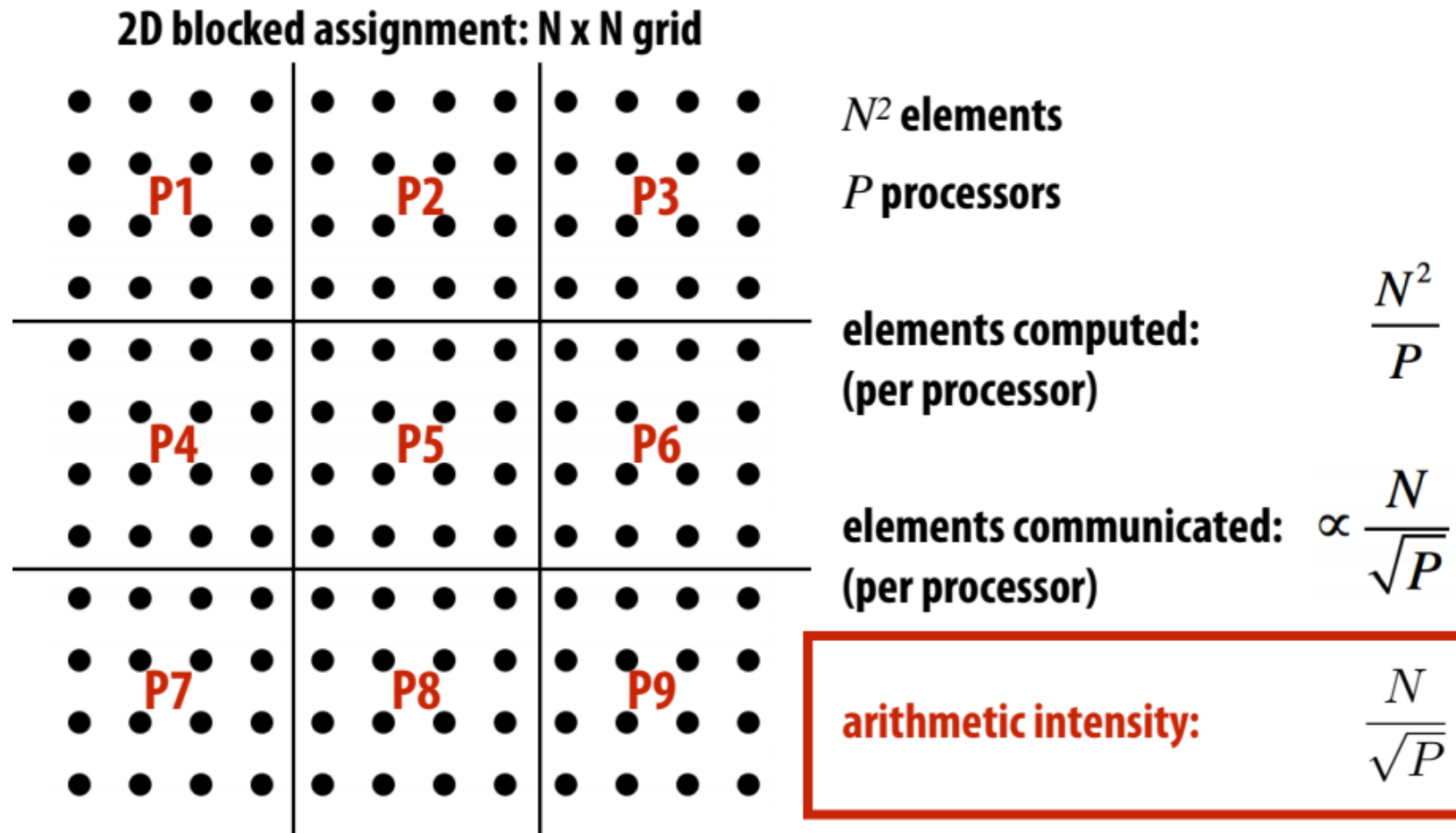
일반적인 함정: 병렬 프로그램 속도 향상과 하나의 코어에서 실행되는 병렬 알고리즘을 비교.(보기 좋게 만들기가 더 쉽음)

Speedup of solver application: 258 x 258 grid

Execution on 32 processor SGI Origin 2000



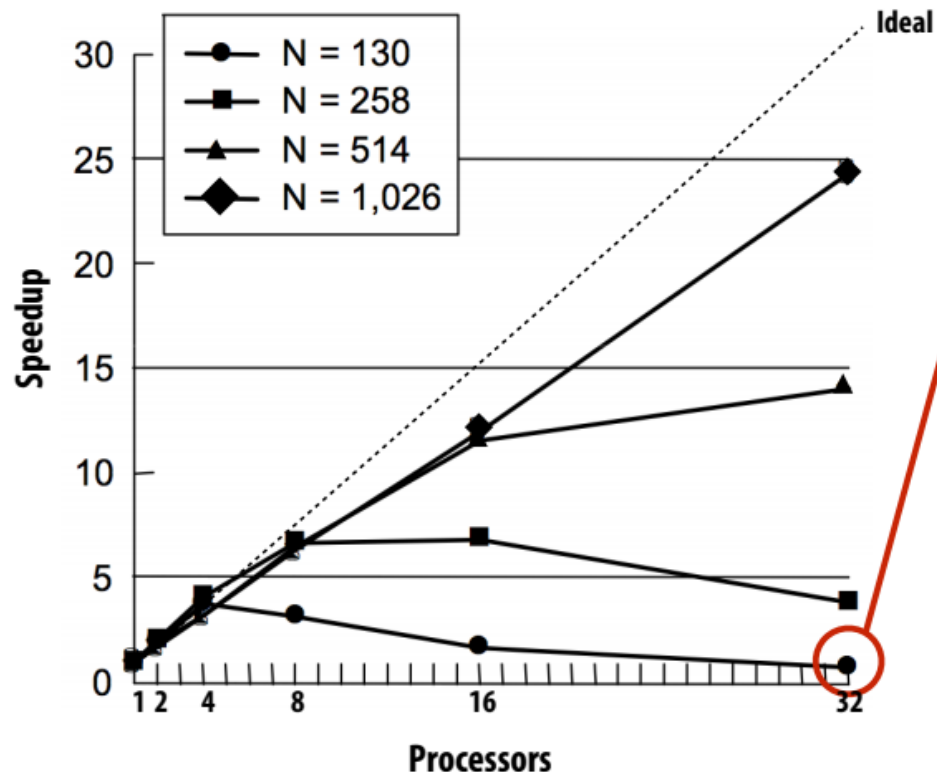
Remember: work assignment in solver



Small N (or large P) yields low arithmetic intensity!

Pitfalls of fixed problem size speedup analysis

Solver execution on 32 processor SGI Origin 2000



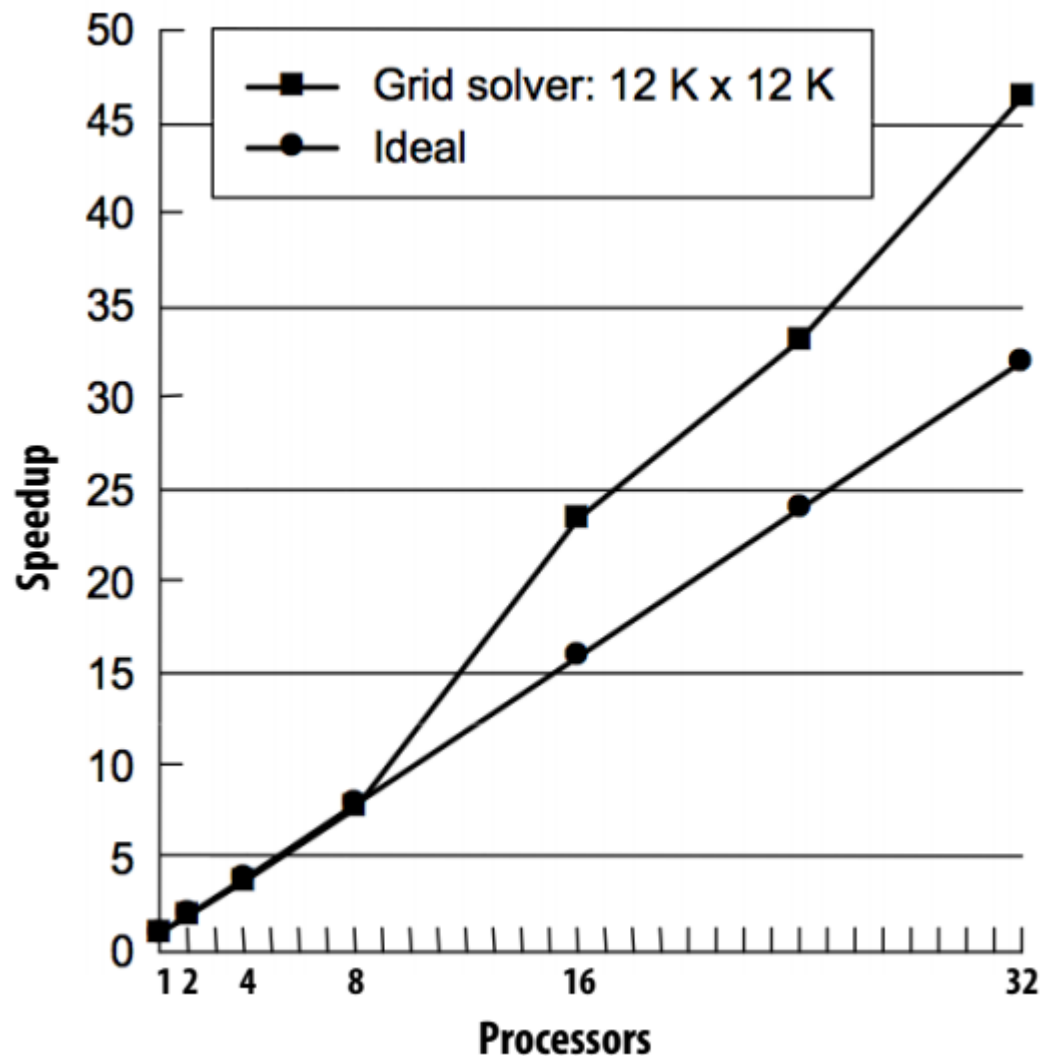
- 혜택 없음! (약간의 속도 저하)
- 문제 크기가 기계에 비해 너무 작음.
(큰 통신 대 연산 비율)
- 작은 문제의 성능을 확장하는 것은 어차피 그다지 중요하지 않을 수 있습니다 (이미 단일 코어에서 충분히 빠르게 실행될 수 있음).

258 x 258 grid on 32 processors: ~ 310 grid cells per processor

1K x 1K grid on 32 processors: ~ 32K grid cells per processor

Pitfalls of fixed problem size speedup analysis

Solver execution on 32 processor SGI Origin 2000



여기: 초선형 속도 향상! 프로세서가 충분하면 각 프로세서에 할당된 그리드 청크가 캐시에 들어가기 시작함(주요 작업 세트는 프로세서별 캐시에 들어갑니다).

또 다른 예: 문제 크기가 단일 머신에 비해 너무 크면, 작업 세트가 메모리에 맞지 않아 디스크에 쓰레싱을 일으킬 수 있음.

(이렇게 하면 더 많은 메모리를 가진 더 큰 병렬 머신의 속도 향상이 놀랍게 보일 것임!).

Understanding scaling

- 해결해야 할 문제의 크기와 병렬 컴퓨터의 크기 사이에는 복잡한 상호 작용이 있을 수 있음.
 - 로드 밸런스, 오버헤드, 산술 강도, 데이터 액세스의 로컬리티에 영향을 미칠 수 있음.
 - 효과는 극적이고 애플리케이션에 따라 달라질 수 있음.
- 고정된 문제 크기로 시스템을 평가하는 것은 문제가 될 수 있음.
 - 너무 작은 문제:
 - ✓ 병렬 처리 오버헤드가 병렬 처리 이점을 압도함(심지어 속도 저하를 초래할 수도 있음)
 - ✓ 문제 크기는 작은 머신에는 적합할 수 있지만 큰 머신에는 부적절할 수 있음.
(대형 머신의 현실적인 사용량을 반영하지 않음!)
 - 문제가 너무 큰 경우:(대형 머신에 적합하도록 문제 크기가 선택됨)
 - ✓ 키 작업 세트가 소형 기계에 "적합"하지 않을 수 있음.
 - ✓ (디스크에 쓰레싱이 발생하거나 키 작업 세트가 캐시 용량을 초과하거나 전혀 실행할 수 없음)
 - ✓ 문제 작업 세트가 대형 머신에는 "맞지만" 소형 머신에는 맞지 않는 경우, 초선형 속도 향상이 발생할 수 있음.
- 머신 크기가 커짐에 따라 문제 크기를 확장하는 것이 바람직할 수 있음.
(같은 문제를 더 빨리 계산하는 것이 아니라 더 많은 계산을 위해 더 큰 머신을 구입)

Summary of tips

- 측정, 측정, 측정...
- 프로그램에 대한 높은 워터마크 설정
 - 컴퓨팅, 동기화 또는 대역폭에 제한이 있나요?
- 스케일링 문제에 유의하세요. 문제가 머신에 잘 맞습니까?